Software Engineering
Engineering the Development of Software Systems

L05 – Software Engineering in Python

Giancarlo Succi
Dipartimento di Informatica – Scienza e Ingegneria
Università di Bologna

Engineering the Development of Software Systems

- L04:
  - Revision of some fundamentals of Java and of the strategy pattern (and, a very little bit, on C++)
- L05:
  - Software Engineering in Python
- L06:
  - Software Engineering for AI systems in Python

Caveat on Python (1/2)

- Here we will reflect on the concepts we saw on architecture and design looking at the code
- We will focus on a concrete example, which could be used also in the overall course
- We will see many examples of code
- Also very detailed, step-by-step examples
- Our focus will be on the software engineering side, though
- That is in understanding why and how people achieved what they achieved
  - not on hacking solutions

Caveat on Python (2/2)

- The most liked programming languages by the instructor
  1. C++
  2. C
  3. Java
  4. Haskell
  5. SmallTalk
  6. ...
  - ...
  - n-1. ...
  - n. Python
- We use Python because it is becoming a standard
- But we start with a mention of the ancestor

Python for our purposes

- About the language
- Structure of the execution of Python
- Virtual Environment

About the language

- Origin and principles of the language
- Fundamental syntax
- Structure of execution
- String formatting
- Object orientation
- Polymorphism and late binding
- Functions as objects
- Decorators
- Static members
- Structure of the virtual machine

Origin of the language

- The language was conceived by Guido van Rossum at CWI in the '80s, got its first implementation in the early '90, and then was modified and upgraded multiple times
- The latest stable version of Python (at the time of writing these slides) is 3.11.5, released on 5th October 2022
- The language is releases in a "kind of" open source license, the Python Software Foundation License but there are debates about it
- Apparently, Python comes from ABC and SETL (see `https://en.wikipedia.org/wiki/History_of_Python`)
- The instructor sees a strong resemblance of SmallTalk (see the references below)
- **References**:
  - Python vs. Smalltalk: from StackOverflow and from Medium.
  - About Python in wikipedia: `https://en.wikipedia.org/wiki/Python_(programming_language)`, `https://en.wikipedia.org/wiki/History_of_Python` and related pages

Principles of the language

- Multiparadigm (scripting, imperative, object oriented, and functional)
- Dynamically typed
- Garbage collected
- "Batteries included"
- Based on a virtual machine ("kind of" interpreted)
- **References**:
  - About Python in wikipedia: `https://en.wikipedia.org/wiki/Python_(programming_language)`

The Zen of Python (1/2)

- By Tim Peters
  - Beautiful is better than ugly.
  - Explicit is better than implicit.
  - Simple is better than complex.
  - Complex is better than complicated.
  - Flat is better than nested.
  - Sparse is better than dense.
  - Readability counts.
  - Special cases aren't special enough to break the rules.
  - Although practicality beats purity.
  - Errors should never pass silently.
  - Unless explicitly silenced.
- **References**:
  - About the Zen of Python in wikipedia: `https://en.wikipedia.org/wiki/Zen_of_Python`

The Zen of Python (2/2)

- By Tim Peters
  - In the face of ambiguity, refuse the temptation to guess.
  - There should be one-- and preferably only one --obvious way to do it.
  - Although that way may not be obvious at first unless you're Dutch.
  - Now is better than never.
  - Although never is often better than *right* now.
  - If the implementation is hard to explain, it's a bad idea.
  - If the implementation is easy to explain, it may be a good idea.
  - Namespaces are one honking great idea – let's do more of those!
- **References**:
  - About the Zen of Python in wikipedia: `https://en.wikipedia.org/wiki/Zen_of_Python`

Multiparadigm

- We will now explore Python in its 4 paradigms
  - Scripting
  - Imperative
  - Functional
  - Object Oriented
- With such an approach we will cycle over the syntax and the semantics of the language to get a comprehensive view of it and revising the fundamental principles of software engineering

Python as a scripting language

Scripting (1/3)

- Install the Python interpreter: multiple options including
  - Mac: see the guidelines in **Pip Upgrade – And How to Update Pip and Python**
  - Windows: like here: **pyenv for Windows**
- Open the Python interpreter ...
- ... and try :)
- Analysing the scripting perspective of Python exposes us to the fundamental control structures
  - variables, if, blocks via indentation, while, for, collections, iterations

Scripting (2/3)

```
% python3.11
Python 3.11.2 (v3.11.2:878ead1ac1, Feb  7 2023, 10:02:41) [Clang 13.0.0 (clang
    -1300.0.29.30)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello World!")
Hello World!
>>> 4+3
7
>>> x = 7+1
>>> x**2
64
>>>
```

Scripting (3/3)

```
% python3.11
Python 3.11.2 (v3.11.2:878ead1ac1, Feb  7 2023, 10:02:41) [Clang 13.0.0 (clang
    -1300.0.29.30)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> the_world_is_flat = True
>>> if the_world_is_flat:
...     print("Be careful not to fall off!")
...
Be careful not to fall off!
>>> ^D
```

Source: https://docs.python.org/3/tutorial/interpreter.html

Indentation

- If I forget the indentation ...

```
% python3.11
Python 3.11.2 (v3.11.2:878ead1ac1, Feb  7 2023, 10:02:41) [Clang 13.0.0 (clang
    -1300.0.29.30)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> the_world_is_flat = True
>>> if the_world_is_flat:
...  print("Be careful not to fall off!")
  File "<stdin>", line 2
    print("Be careful not to fall off!")
    ^
IndentationError: expected an indented block after 'if' statement on line 1
>>> ^D
```

Source: `https://docs.python.org/3/tutorial/interpreter.html`

Dynamic Typing

- Typing is dynamic and enforced ...

```
% python3.11
Python 3.11.2 (v3.11.2:878ead1ac1, Feb  7 2023, 10:02:41) [Clang 13.0.0 (clang
    -1300.0.29.30)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> x = "a string"
>>> x = 1
>>> print(x)
1
>>> y = 1 + "a string"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
>>>
```

Source: https://docs.python.org/3/tutorial/introduction.html

Math

- Mathematics follows mostly the usual approaches ...

```
% python3.11
>>> 3+4
7
>>> 6-3.4
2.6
>>> 4*7.0
28.0
>>> 7/3
2.3333333333333335
>>> 7//3
2
>>> 7%3
1
>>> 7**3
343
>>> 3 + _
346
>>>
```

Source: `https://docs.python.org/3/tutorial/introduction.html`

Strings (1/2)

- Strings are a bit more peculiar ...

```
>>> x='strings can be single quoted '
>>> y="or double quoted "
>>> x + y + "and  joined with  +  "
'strings can be single quoted or double quoted and joined with +  '
>>>
>>> z=''' strings can span
... multiple lines using triple
...
... quotes'''
>>> z
' strings can span\nmultiple lines using triple\n\nquotes'
>>> print(z)
 strings can span
multiple lines using triple

quotes
>>>
```

Source: `https://docs.python.org/3/tutorial/introduction.html`

Strings (2/2)

- Strings are a bit more peculiar ...

```
>>> w='And I can add \n special chars like in C, which are printed using print'
>>> w
'And I can add \n special chars like in C, which are printed using print'
>>> print(w)
And I can add
 special chars like in C, which are printed using print
>>> c='concatenation ' 'is done just sequencing strings '
>>> c
'concatenation is done just sequencing strings '
>>> c ' but not using variables'
  File "<stdin>", line 1
    c ' but not using variables'
      ^^^^^^^^^^^^^^^^^^^^^^^^^
SyntaxError: invalid syntax
>>> 2*' and the n* operator repeats the string n times'
' and the n* operator repeats the string n times and the n* operator repeats the
     string n times'
```

Source: `https://docs.python.org/3/tutorial/introduction.html`

Indexing Strings (1/2)

- Strings can be indexed ...

```
>>> e='My Example of Python'
>>> e[0]
'M'
>>> e[19]
'n'
>>> e[20]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
>>> e[-1]
'n'
>>> e[-19]
'y'
>>> e[-20]
'M'
>>> e[3:5]
'Ex'
>>> e[-4:-1]
'tho'
```

Indexing Strings (2/2)

- Ranges are start-included, end-excluded ...
- Slicing can include the step as third parameter

```
>>> e[0:3]
'My '
>>> e[11:]
'of Python'
>>> e[:11]+e[11:]
'My Example of Python'
>>> e[2:10:2]
' xml'
>>> e[::2]
'M xml fPto'
>>> e[::-1]
'nohtyP fo elpmaxE yM'
```

Source: `https://docs.python.org/3/tutorial/introduction.html`

More on Strings

- Strings are immutable

```
>>> e[0]='T'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
>>> w = 'T' + e[1:]
>>> w
'Ty Example of Python'
>>>
```

- Length of a string

```
>>> len(w)
20
>>> len("123")
3
```

Source: `https://docs.python.org/3/tutorial/introduction.html`

String Interpolation (1/9)

- **String interpolation** is the process of inserting variables or expressions into a string
- It allows the dynamic construction of strings using placeholders
- Useful for formatting messages, reports, logs, etc.

```
name = "Alice"
age = 30

# Example of string interpolation
print("My name is {} and I'm {}.".format(name, age))
```

- Python supports several interpolation methods:
  - Modulo operator (%)
  - `str.format()` method
  - f-strings (since Python 3.6)

Source: `https://realpython.com/python-f-strings/`

String Interpolation (2/9)

- % is one of the earliest ways to format strings in Python
- Uses % followed by a format specifier

```
name = "Jane"
age = 25

# Interpolates a single string variable using %s
print("Hello, %s!" % name)

# Interpolates another single variable (an integer in this case)
# %s automatically converts any type to string
print("You're %s years old." % age)

# Interpolates multiple variables using a tuple of values
# The number of placeholders must match the number of items
print("Hello, %s! You're %s." % (name, age))

# Interpolates values using a dictionary
# Keys in the dictionary must match those in the format string
print("Hello, %(name)s!" % {"name": "Jane"})
```

Source: https://realpython.com/python-f-strings/

String Interpolation (3/9)

- % operator cannot interpolate tuples directly
- Hard to read and error-prone syntax
- Limited formatting capabilities

```
# Problematic case: multiple values passed to a single %s placeholder
print("Data: %s" % ("John", 35))
# -> TypeError: not all arguments converted during string formatting

# Fixed: wrap the tuple in another tuple to treat it as a single object
print("Data: %s" % (("John", 35),))     # Output: Data: ('John', 35)

# Custom formatting: format float to 2 decimal places
print("Balance: $%.2f" % 5425.9292)     # Output: Balance: $5425.93

# Right-align an integer in a field 5 characters wide
print("Age: %5s" % 35)                  # Output: Age:    35
```

- Use with caution in coding!
- Prefer `str.format()` or f-strings

Source: `https://realpython.com/python-f-strings/`

String Interpolation (4/9)

- `str.format()` method was introduced in Python 2.6 / 3.0
- Uses curly braces {} for placeholders
- Supports positional and keyword arguments

```
name = "Jane"
age = 25
# Positional arguments: values inserted in order of appearance
print("Hello, {}! You're {}.".format(name, age))

# Indexed arguments: specify the position of arguments manually
print("Hello, {1}! You're {0}.".format(age, name))

# Keyword arguments: use named placeholders for clarity
print("Hello, {name}! You're {age}.".format(name="Jane", age=25))

# Dictionary unpacking: pass key-value pairs using ** operator
person = {"name": "Jane", "age": 25}
print("Hello, {name}! You're {age}.".format(**person))

# Output: Hello, Jane! You're 25.
```

Source: `https://realpython.com/python-f-strings/`

String Interpolation (5/9)

- String formatting with `.format()` supports formatting mini-language (precision, alignment, padding)

```
# Float precision
print("Balance: ${:.2f}".format(5425.9292))  # $5425.93

# String centering and padding
print("{:=^30}".format("Centered"))          # Centered with "="

# Right-aligned integers
print("{:>5}".format(42))                    # "   42"
```

- Flexible and more readable than % operator
- Still used, though f-strings are preferred today

Source: `https://realpython.com/python-f-strings/`

String Interpolation (6/9)

- F-strings were introduced in Python 3.6 via PEP 498
- Also called **formatted string literals**
- Start with `f` or `F` before the string
- Embed variables or expressions directly in curly braces {}

```
name = "Jane"
age = 25

print(f"Hello, {name}! You're {age} years old.")
# Output: Hello, Jane! You're 25 years old.
```

- More concise and readable than `%` and `.format()`
- Variables must be in scope at runtime

Source: `https://realpython.com/python-f-strings/`

String Interpolation (7/9)

- F-strings support full Python expressions
- Expressions are evaluated at runtime

```
# Simple arithmetic
print(f"{2 * 21}")                          # Output: 42

# Method call
name = "Jane"
print(f"Uppercased: {name.upper()}")        # Output: JANE

# List comprehension
print(f"{[2**n for n in range(3, 9)]}")     # Output: [8, 16, 32, 64, 128, 256]
```

- You can embed functions, method calls, comprehensions, etc.

Source: `https://realpython.com/python-f-strings/`

String Interpolation (8/9)

- F-strings support the formatting mini-language
- Format specifiers follow a colon : after the expression

```
balance = 5425.9292
print(f"Balance: ${balance:.2f}")
# Output: Balance: $5425.93

heading = "Centered string"
print(f"{heading:=^30}")
# Output: =======Centered string========
```

- Format strings for precision, alignment, padding
- Same syntax as `.format()` and `format()` function

Source: `https://realpython.com/python-f-strings/`

String Interpolation (9/9)

Method	Syntax	Key Features
% operator	`"Hello, %s!" % name`	Limited formatting; error-prone with tuples
`.format()`	`"Hello, {}!".format(name)`	More powerful; supports positional/keyword args and formatting mini-language
F-strings	`f"Hello, {name}!"`	Most readable; supports expressions; fast; formatting mini-language

- Prefer **f-strings** for modern Python code due to simplicity, performance, and power
- Use `.format()` when compatibility with Python ≤ 3.6 is needed
- Avoid % operator for new code

Source: `https://realpython.com/python-f-strings/`

Collections

- Python does not have arrays
- Python has 4 basic collection types, partly with syntax similar to that of strings:
  - Lists: ordered, mutable collections with duplicates
  - Tuples: ordered, immutable collection with duplicates
  - Sets: unordered collection, without duplications, whose members are immutable
  - Dictionaries: ordered, mutable collections without duplicates

Source: `https://www.w3schools.com/python/python_tuples.asp`

Lists (1/8)

- List are ordered collections of elements, with a syntax resembling that of strings

```
>>> odd = [1, 3, 5, 7, 9]
>>> odd
[1, 3, 5, 7, 9]
>>> odd[0]
1
>>> odd[4]
9
>>> odd[-1]
9
>>> odd[-5]
1
>>> odd[1:3]
[3, 5]
>>> odd[:2]+odd[2:]
[1, 3, 5, 7, 9]
>>> len(odd)
5
```

Source: https://docs.python.org/3/tutorial/introduction.html

Lists (2/8)

- List are mutable, append-able, slice-able

```
>>> odd[2]=20
>>> odd
[1, 3, 20, 7, 9]
>>> odd.append(11)
>>> odd
[1, 3, 20, 7, 9, 11]
>>> odd[3:5]=[100,200,300]
>>> odd
[1, 3, 20, 100, 200, 300, 11]
>>> odd[1:3] = []
>>> odd
[1, 100, 200, 300, 11]
>>> odd[:] = []
>>> odd
[]
```

Source: `https://docs.python.org/3/tutorial/introduction.html`

Lists (3/8)

- List are nestable and heterogeneous

```
>>> a = [1,2,3]
>>> b = [10,20]
>>> c = [90, 800, -34]
>>> n = [a, b, c]
>>> n
[[1, 2, 3], [10, 20], [90, 800, -34]]
>>> n[2][1]
800
>>> z = [1, "xxx", 3]
>>> z
[1, 'xxx', 3]
>>> z[1]
'xxx'
```

Source: https://docs.python.org/3/tutorial/introduction.html

Lists (4/8)

- Copy of references

```
>>> x = [1, 2, 3, 4]
>>> y = x
>>> y[2]=200
>>> x
[1, 2, 200, 4]
```

Source: https://www.geeksforgeeks.org/copy-python-deep-copy-shallow-copy/

Lists (5/8)

- Shallow copies

```
>>> w = x.copy()
>>> w[1]=350
>>> w
[1, 350, 200, 4]
>>> x
[1, 2, 200, 4]
```

Source: https://www.geeksforgeeks.org/copy-python-deep-copy-shallow-copy/

Lists (6/8)

- Shallow copies (more)

```
>>> z = [11,22,33]
>>> k = [121, 132, 143, 154]
>>> j = [z,k]
>>> j
[[11, 22, 33], [121, 132, 143, 154]]
>>> i = j.copy()
>>> k[1] = 222
>>> i
[[11, 22, 33], [121, 222, 143, 154]]
>>> i[0] = [5, 8, 13]
>>> k[1] = 222
>>> i
[[5, 8, 13], [121, 222, 143, 154]]
>>> j
[[11, 22, 33], [121, 222, 143, 154]]
```

Source: `https://www.geeksforgeeks.org/copy-python-deep-copy-shallow-copy/`

Lists (7/8)

- Deep copies

```
>>> j
[[11, 22, 33], [121, 222, 143, 154]]
>>> import copy
>>> dc = copy.deepcopy(j)
>>> k[1] = 423
>>> j
[[11, 22, 33], [121, 423, 143, 154]]
>>> dc
[[11, 22, 33], [121, 222, 143, 154]]
```

- Membership

```
>>> aList = [1, 2, 3, 5, 0, 9, 0, 7, 1, 5]
>>> 3 in aList
True
>>> 12 in aList
False
```

Source: `https://www.geeksforgeeks.org/copy-python-deep-copy-shallow-copy/`

Lists (8/8)

- Deletion

```
>>> aList = [1, 2, 3, 5, 0, 9, 0, 7, 1, 5]
>>> aList.pop(2)
3
>>> aList
[1, 2, 5, 0, 9, 0, 7, 1, 5]
>>> aList.pop(-3)
7
>>> aList
[1, 2, 5, 0, 9, 0, 1, 5]
>>> aList.remove(5)
>>> aList
[1, 2, 0, 9, 0, 1, 5]
>>> del aList[1]
>>> aList
[1, 0, 9, 0, 1, 5]
>>> aList.clear()
>>> aList
[]
```

Source: `https://note.nkmk.me/en/python-list-clear-pop-remove-del//`

Tuples (1/5)

- Tuples are ordered, immutable collections of elements, with syntax resembling arrays

```
>>> aTuple=("touple", 2, 9, [10, 2, 3], "start")
>>> aTuple
('touple', 2, 9, [10, 2, 3], 'start')
>>> aTuple[0]
'touple'
>>> aTuple[-1]
'start'
>>> aTuple[2:]
(9, [10, 2, 3], 'start')
>>> aTuple[1] = "tryToChange"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> aTuple.append(3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'tuple' object has no attribute 'append'
```

Source: https://www.w3schools.com/python/python_tuples.asp/

Tuples (2/5)

- Tuples are immutable!

```
>>> anAlias=aTuple
>>> aTuple = aTuple + (1, 2, 3)
>>> aTuple
('touple', 2, 9, [10, 2, 3], 'start', 1, 2, 3)
>>> anAlias
('touple', 2, 9, [10, 2, 3], 'start')
>>> aTuple = aTuple + (3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate tuple (not "int") to tuple
>>> aTuple = aTuple + (3,)
>>> aTuple
('touple', 2, 9, [10, 2, 3], 'start', 1, 2, 3, 3)
>>> len(aTuple)
9
>>> 2 in aTuple
True
>>> 12 not in aTuple
True
```

Tuples (3/5)

- Tuples are immutable ... but be careful of references!

```
>>> a = [3,4,5]
>>> b=(a,6)
>>> b
([3, 4, 5], 6)
>>> a[0]=1
>>> b
([1, 4, 5], 6)
>>> c = b
>>> a[1]=10
>>> b
([1, 10, 5], 6)
>>> c
([1, 10, 5], 6)
>>> c = b.copy()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'tuple' object has no attribute 'copy'
```

Source: https://www.w3schools.com/python/python_tuples.asp/

Tuples (4/5)

- Tuples are immutable ... but be careful of references!

```
>>> import copy
>>> d = copy.deepcopy(b)
>>> d
([1, 10, 5], 6)
>>> a[2]=8
>>> b
([1, 10, 8], 6)
>>> c
([1, 10, 8], 6)
>>> d
([1, 10, 5], 6)
>>>
```

Source: https://www.w3schools.com/python/python_tuples.asp/

Tuples (5/5)

- Shortcuts and conversions

```
>>> anAlias=aTuple
>>> aTuple += (55,)
>>> aTuple
('touple', 2, 9, [10, 2, 3], 'start', 1, 2, 3, 3, 55)
>>> anAlias
('touple', 2, 9, [10, 2, 3], 'start', 1, 2, 3, 3)
>>> aList = [1, 2, 3]
>>> aConvertedTuple = tuple(aList)
>>> aList
[1, 2, 3]
>>> aConvertedTuple
(1, 2, 3)
>>> aList[2]=5
>>> aList
[1, 2, 5]
>>> aConvertedTuple
(1, 2, 3)
```

Source: https://www.w3schools.com/python/python_tuples_update.asp/

Sets (1/3)

- Sets are unordered collections without duplicates whose elements cannot change

```
>>> aSet = {1, 2, "red"}
>>> aSet
{1, 2, 'red'}
>>> anotherSet = {3, 4, "green", 4, "green"}
>>> anotherSet
{3, 4, 'green'}
>>> 1 in aSet
True
>>> "green" in aSet
False
>>> "green" in anotherSet
True
>>> oneIsLikeTrue = { 1, True, "green"}
>>> oneIsLikeTrue
{1, 'green'}
>>> len(oneIsLikeTrue)
2
```

Source: https://www.w3schools.com/python/python_sets.asp/

Sets (2/3)

```
>>> aThirdSet = aSet | anotherSet
>>> aThirdSet
{1, 2, 3, 4, 'green', 'red'}
>>> aFourthSet = aThirdSet & {2, 3, 200, 'green'}
>>> aFourthSet
{2, 3, 'green'}
>>>
>>> aSetFromAList = set([9, 8, 7])
>>> aSetFromAList
{8, 9, 7}
>>> aSetFromATuple = set((6, 5, 4))
>>> aSetFromATuple
{4, 5, 6}
>>> aSetFromATuple + aSetFromAList
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'set' and 'set'
```

Source: https://www.w3schools.com/python/python_sets.asp/

Sets (3/3)

- Mutable elements like lists, the same sets, and dictionaries cannot be part of sets

```
>>> a = {3, 1, 5, 4}
>>> b = [10, 11, 12]
>>> a.add(b)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'list'
>>> c = (10, 11, 12)
>>> a.add(c)
>>> a
{1, 3, 4, 5, (10, 11, 12)}
>>> d = {3, 4, 1}
>>> a.add(d)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'set'
```

Source: https://www.w3schools.com/python/python_sets.asp/

Dictionaries (1/2)

- Dictionaries are ordered collections of pair (key:value), indexed by key, where each key can appear only once

```
>>> aDictionary = { "breed":"Dog", "weight":8, "name":"Deimon", "age":3}
>>> aDictionary
{'breed': 'Dog', 'weight': 8, 'name': 'Deimon', 'age': 3}
>>> aDictionary["breed"]
'Dog'
>>> len(aDictionary)
4
>>> aDictionary.keys()
dict_keys(['breed', 'weight', 'name', 'age'])
>>> aDictionary.values()
dict_values(['Dog', 8, 'Deimon', 3])
>>> aDictionary.update({"weight":7.5,"color":"blenheim"})
>>> aDictionary
{'breed': 'Dog', 'weight': 7.5, 'name': 'Deimon', 'age': 3, 'color': 'blenheim'}
```

Source: https://www.w3schools.com/python/python_dictionaries.asp/

Dictionaries (2/2)

- Dictionaries can be largely manipulated

```
>>> aDictionary.popitem()
('color', 'blenheim')
>>> aDictionary
{'breed': 'Dog', 'weight': 7.5, 'name': 'Deimon', 'age': 3}
>>> aDictionary.pop('weight')
7.5
>>> aDictionary
{'breed': 'Dog', 'name': 'Deimon', 'age': 3}
>>> del aDictionary["age"]
>>> aDictionary
{'breed': 'Dog', 'name': 'Deimon'}
```

Source: https://www.w3schools.com/python/python_dictionaries_remove.asp/

The range function

- The range function is used to generate lists of numbers for iterations
- It returns a range object, which could then be converted in a list for printing

```
>>> list(range(3))
[0, 1, 2]
>>> list(range(-4, 5))
[-4, -3, -2, -1, 0, 1, 2, 3, 4]
>>> list(range(2, 20, 3))
[2, 5, 8, 11, 14, 17]
>>> list(range(2, 20, 7))
[2, 9, 16]
>>> list(range(0))
[]
>>> range(4.3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'float' object cannot be interpreted as an integer
```

Source: https://thepythonguru.com/python-builtin-functions/range/

Control flow – `if`

- The `if` statement is all based on indentation

```
>>> import math
>>> a = 4
>>> b = 5
>>> c = -1
>>> delta = b**2 - 4*a*c
>>> if delta > 0:
...     x1 = (-b - math.sqrt(delta))/(2*a)
...     x2 = (-b + math.sqrt(delta))/(2*a)
... elif delta == 0:
...     x1 = x2 = - b / (2*a)
... else:
...     x1 = complex(-b, math.sqrt(-delta)/(2*a))
...     x2 = complex(-b, math.sqrt(-delta)/(2*a))
...
>>> x1
-1.425390529679106
>>> x2
0.17539052967910607
>>>
```

Control flow – `while`

- The `while` statement is the typical top-tested loop based on indentation

```
>>> i = 5
>>> factorial = 1
>>> while i > 1:
...  factorial *= i
...  i -= 1
... else:
...  print("the result is ",factorial)
...
the result is 120
>>>
```

Source: `https://docs.python.org/3/tutorial/controlflow.html/`

Flow in loops

- Python has three construct to manage the flow in loops:
  - `break`: like in C and Java breaks the loop (the `else` statement is not executed)
  - `continue`: like in C and Java, suspends the current iteration and jumps directly to the next one
  - `pass`: absent in C and Java, moves to the next block
- Now we analyse in details the following snippets

Source: `https://docs.python.org/3/tutorial/controlflow.html/`

break and continue

```
i = 0
print("break")
while i in range(10):
    i += 1
    if i == 5:
        print("i is 5!")
        break
        print("break: I should not get here!")
    print("Standard printout for iteration ",i)
else:
    print("End of loop break")
i = 0
print("continue")
while i in range(10):
    i += 1
    if i == 5:
        print("i is 5!")
        continue
        print("continue: I should not get here!")
    print("Standard printout for iteration ",i)
else:
    print("End of loop continue")
```

pass

```
i = 0
print("pass")
while i in range(10):
    i += 1
    if i == 5:
        print("i is 5!")
        pass
        print("pass: I should not get here!")
    print("Standard printout for iteration ",i)
else:
    print("End of loop pass")
```

Output (1/2)

```
break
Standard printout for iteration  1
Standard printout for iteration  2
Standard printout for iteration  3
Standard printout for iteration  4
i is 5!
continue
Standard printout for iteration  1
Standard printout for iteration  2
Standard printout for iteration  3
Standard printout for iteration  4
i is 5!
Standard printout for iteration  6
Standard printout for iteration  7
Standard printout for iteration  8
Standard printout for iteration  9
Standard printout for iteration  10
End of loop continue
```

Output (2/2)

```
pass
Standard printout for iteration  1
Standard printout for iteration  2
Standard printout for iteration  3
Standard printout for iteration  4
i is 5!
pass: I should not get here!
Standard printout for iteration  5
Standard printout for iteration  6
Standard printout for iteration  7
Standard printout for iteration  8
Standard printout for iteration  9
Standard printout for iteration  10
End of loop pass
```

`iterator` (1/2)

- This is a case when superclasses are very useful
- `iterator` is a class supporting iterations over collections, that is, referring to a collection of objects, sequentializing and indexing them, and with a method `__next__()` that:
  - returns one by one the elements of the object
  - updaties the state of the `iterator` so that it always refers to *next* element in the sequence
  - raises an exception `StopIteration` when there are no more elements to point to
- Please notice the mix of scripting and object orientation

Source: `https://stackoverflow.com/questions/9884132/what-are-iterator-iterable-and-iteration#:~:text=An%20iterable%20is%20a%20object,__next__()%20in%203/`

iterator (2/2)

```
>>> string = "iter"
>>> stringIterator = iter(string)
>>> next(stringIterator)
'i'
>>> next(stringIterator)
't'
>>> stringIterator.__next__()
'e'
>>> stringIterator.__next__()
'r'
>>> stringIterator.__next__()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
>>>
```

Source: https://stackoverflow.com/questions/9884132/what-are-iterator-iterable-and-iteration#:~:text=An%20iterable%20is%20a%20object,__next__()%20in%203/

iterable

- An `iterable` is the base class for anything that can be iterated over
- It has a method `__iter__()` that returns an `iterator`
- This is also what is done by the global function `iter()` (see slide 61)
- Notice the duality global functions and member functions, like for the global function `next()` and member function `__next__()`
- The duality `iterable` – `iterator` is heavily used in `for` loops, list comprehension etc (see later)

```
>>> string = "iter"
>>> stringIterator = string.__iter__()
>>> next(stringIterator)
'i'
```

Source: `https://stackoverflow.com/questions/9884132/what-are-iterator-iterable-and-iteration`

for loop

- The `for` loop in Python iterates over an `iterator`
- At every step in the loop there is an implicit call to the `__next__()` member function of the iterator
- At the last step there is an implicit catch of the `StopIteration` exception
- `else`, `break`, `continue`, and `pass` work like in a `while` loop

```
>>> string = "iter"
>>> for i in string:
...     print(i)
... else:
...     print("end of string")
...
i
t
e
r
end of string
```

Source: `https://wiki.python.org/moin/ForLoop`

Command line (1/2)

- Command line arguments work like in Java with `argv`
- In addition there is the function `getopt` that helps parsing the arguments one by one

```
import sys, getopt
print("Argument line: ",sys.argv)
print("Number of arguments: ",len(sys.argv))
i=0
for arg in sys.argv:
        print("argument ",i," : ",arg)
        i += 1
options, arguments = getopt.getopt(sys.argv[1:],"nh:o:")
for opt, val in options:
        if opt=="-h" : print ("Help: ",val)
        elif opt=="-o" : print ("Other: ",val)
        elif opt=="-n" : print ("n means ", end=""); print("\t no arguments")
        else : print(opt, " and ", val, " are not acceptable")
print("The other arguments are ", arguments)
```

Source: `https://docs.python.org/3/library/getopt.html`

Command line (2/2)

- The output is:

```
% python3 commandLine.py -o xxx -h help -n a b c
Argument line:  ['commandLine.py', '-o', 'xxx', '-h', 'help', '-n', 'a', 'b', 'c
    ']
Number of arguments:  9
argument  0  :  commandLine.py
argument  1  :  -o
argument  2  :  xxx
argument  3  :  -h
argument  4  :  help
argument  5  :  -n
argument  6  :  a
argument  7  :  b
argument  8  :  c
Other:  xxx
Help:  help
n means   no arguments
The other arguments are  ['a', 'b', 'c']
```

Source: `https://docs.python.org/3/library/getopt.html`

Python as an imperative language

Functions (1/4)

- In Python there are functions with parameters and return values

```
# exampleFunction.py
import math
def secondOrderEquation(a, b= 3, c= 0.5):
   delta = b*2 -4*a*c
   x1 = (-b - math.sqrt(delta))/(2*a)
   x2 = (-b + math.sqrt(delta))/(2*a)
   return a, b, c, x1, x2

print(secondOrderEquation(1, 6, 1))
print(secondOrderEquation(1, 6))
print(secondOrderEquation(1))
print(secondOrderEquation(1, c=-1))

% python3 exampleFunction.py
(1, 6, 1, -4.414213562373095, -1.5857864376269049)
(1, 6, 0.5, -4.58113883008419, -1.4188611699158102)
(1, 3, 0.5, -2.5, -0.5)
(1, 3, -1, -3.08113883008419, 0.08113883008418976)
```

Functions (2/4)

- Parameters are all passed by values and values can be references, as in Java
- But there are mutable and immutable objects!
- Functions can be parameters of other functions

```
# higherOrder.py
def apply(f,a):
    return f(a)
def increment(x):
    return x+1
def square(x):
    return x**2
def main():
    x = apply(increment,3)
    y = apply(square,4)
    print(x, y)
if __name__ == "__main__":
    main()
% python3 higherOrder.py
4 16
```

Functions (3/4)

- There are anonymous functions
- They are called `lambda` as they resemble lambda expressions

```
# exampleLambda.py
def apply(f,a):
    return f(a)
def main():
    x = apply(lambda x: x+1,3)
    y = apply(lambda x: x**2,4)
    print(x, y)
if __name__ == "__main__":
    main()
% python3 higherOrder.py
4 16
```

Functions (4/4)

- There is a *kind of* `main` function
- Every module has an internal variable, `__name__`
  - this value is set to `__main__` if the module is the one invoked directly in the command line
- Otherwise, it is set to the *name of the module*, that is, the name of the file without the suffix `.py`
- There is a convention to call a function `main()` as the first function to be executed by a module that is directly called in the command line

```
if __name__ == "__main__":
    main()
```

Nested functions

- Functions can be nested, like in Pascal

```
# nested.py

def outer(x) :
    y = 10
    def inner(z):
        return z + y
    w = inner(x)
    return w

if __name__ == "__main__":
    print(outer(3))

% python3 nested.py
13
```

Scoping rules (1/5)

- LEGB scoping
  1. Local
  2. Enclosing
  3. Global
  4. Builtin
- The keyword `global` declares a variable as global
- The keyword `nonlocal` declares a local variable in an inner scope associated to the outer scope

Scoping rules (2/5)

- Without a `global` or a `nonlocal` declaration:
  - if a variable is initialized with a name already used in an outer scope, then it is treated as a new local variable, which then hides the global one
  - if a variable is used with a name already used in an outer scope, but
    - then if its value is then changed inside the scope
    - an error is raised, as
    - it is treated as a local variable that previously was used without being initialized.

Scoping rules (3/5)

```
# scoping.py
x = 5
def testScope(a):
    y = x + a
    global z
    z = y + 1
    return y + z
def main():
    print (testScope(5) + z)
if __name__ == "__main__" :
    main()
% python3 scoping.py
32
```

Scoping rules (4/5)

```
# nonLocalScoping.py
x = 1; y = 2; w = 3; z = 4
def testScope():
   y = 20; w = 30; k = z;
   def testInnerScope():
      w = 300
      global x
      nonlocal y
      x = 100; y = 200; z = 400
      print("testInnerScope: x=",x,",y=",y,",w=",w,",z=",z)
   testInnerScope()
   print("testScope: x=",x,",y=",y,",w=",w,",z=",z)
def main() :
   testScope()
   print("main: x=",x,", y=",y,", w=",w,", z=",z)
if __name__ == "__main__" :
   main()
% python3.11 nonLocalScoping.py
testInnerScope: x= 100 ,y= 200 ,w= 300 ,z= 400
testScope: x= 100 ,y= 200 ,w= 30 ,z= 4
main: x= 100 ,y= 2 ,w= 3 ,z= 4
```

Scoping rules (5/5)

```
# simpleScopingError.py
x = 1
y = 2
z = 3
w = 0
def testScope():
    global x
    y = 4
    z = 5 + w
    global z
    w = -1
def main() :
    print("in main x =",x,", y =",y)

global z
^^^^^^^^
SyntaxError: name 'z' is assigned to before global declaration
z = 5 + w
        ^
UnboundLocalError: cannot access local variable 'w' where it is not associated
    with a value
```

Default parameters as static (1/3)

- Default parameters are evaluated only once, at the first call
  - they are like static local variables in C
  - normally objects are immutable
  - but when objects are mutable something unexpected may happen

Default parameters as static (2/3)

```
# defaultParametersStatic.py
i = 1
def f(a=[50]):
    global i
    print("Call ", i, "At the beginning of f a is:",a)
    a[0] +=1
    print("At the end of f a is:",a)
    i+=1
def main():
    x = [2]
    print("Before the first call to f x is: ",x)
    f(x)
    print("After the first call to f x is: ",x)
    f()
    print("After the second call to f x is: ",x)
    f()
    print("Before the third call to f x is: ",x)
    f(x)
    print("Before the third call to f x is: ",x)
if __name__=="__main__":
    main()
```

Default parameters as static (3/3)

```
% python3.11 defaultParametersStatic.py
Before the first call to f x is:  [2]
Call  1 At the beginning of f a is: [2]
At the end of f a is: [3]
After the first call to f x is:  [3]
Call  2 At the beginning of f a is: [50]
At the end of f a is: [51]
After the second call to f x is:  [3]
Call  3 At the beginning of f a is: [51]
At the end of f a is: [52]
Before the third call to f x is:  [3]
Call  4 At the beginning of f a is: [3]
At the end of f a is: [4]
Before the third call to f x is:  [4]
```

Modules (1/3)

- Python structure the code quite like C
- Every file is a module
  - the name of the module is the name of the file without the extension `.py`
  - it is stored in the variable `__name__`
  - such name is changed to `__main__` if the module is the one directly invoked
- A module is an object in Python

Source: `https://docs.python.org/3/tutorial/modules.html`

Modules (2/3)

- To access the module you need to specify the instruction `import amodule`
  - where `amodule.py` is the file where the module is located
  - such file must reside in a location specified by the environmental variable `PYTHONPATH`
  - and any name to use from such module has to be prefixed by the name of the module
  - that is, to use function `goofy()`, the full name `amodule.goofy()` must be specified

Source: `https://docs.python.org/3/tutorial/modules.html`

Modules (3/3)

- To load the names of the module inside the current namespace: `from amodule import goofy`
- To import all the names inside `amodule.py`: `from amodule import *`, in which case only the names starting with an _ will not be directly accessible
  - in this way it is possible just to call `goophy()`
- Be careful, but is also possible to rename an entity or module as
  - `import amodule as unmodulo`, leading to calls like `unmodulo.goophy()` or even
  - `from amodule import goofy as pippo`, leading to calls like `amodule.pippo()`

Source: `https://docs.python.org/3/tutorial/modules.html`

Namespaces

- A namespace is the set of all names associated to objects visible at a certain time and place
- The current namespace is accessible with the instruction `dir()`
- It is also possible to view the namespace of a specific module with `dir(amodule)`

Source: `https://py-pkgs.org/04-package-structure.html`

Packages (1/2)

- A package is a collection of modules
- From a system viewpoint, a package is a directory:
  - inside the list specified by `sys.path`
  - with a `__init__.py` file, which can also be empty but must exist
- From an internal perspective, a package is a module object
- The `import` statement applies for packages like for modules
- A subpackage is a directory of a (sub)packages with a with a `__init__.py` file
- The `.` separates a package from subpackages and modules

Source: `https://stackoverflow.com/questions/4881897/python-project-and-package-directories-layout` and `https://py-pkgs.org/04-package-structure.html`

Packages (2/2)

- When a package is loaded, its `__init__.py` is executed
- It acts as a kind of constructor of the package
- packages can contain references to other packages using absolute or relative paths
- using relative paths
  - the `.` refers to the current directory
  - the `..` refers to the parent directory

Source: `https://stackoverflow.com/questions/4881897/python-project-and-package-directories-layout` and `https://py-pkgs.org/04-package-structure.html`

Importing elements (1/3)

- `import` is the statement defines the connection between the current module to the one containing the entities we are interested
- when the statement `import A` is executed:
  - the runtime looks for
    - a file named `A.py` (a module) or
    - a directory named `A` with a file `__init__.py` (a packages)

    in a list of directories specified in the `sys.path` variable
- if the results is a module, then the names inside it become visible
- if the result is a package, then the names inside it become visible and `__init__.py` is executed

Source: `https://chrisyeh96.github.io/2017/08/08/definitive-guide-python-imports.html`

Importing elements (2/3)

- The `import` directive has many variants
  - `import <module or package>`
  - `from <package> import <module or (sub)package or object>`
  - `from <module> import <object>`
- *in Python functions are objects!*
- The structure of the naming after an `import` of `X` is:
  - if `X` is a module or a package:
    - `X.entity`
  - if `X` is a variable:
    - `X` is used as is

Source: `https://chrisyeh96.github.io/2017/08/08/definitive-guide-python-imports.html`

Importing elements (3/3)

- if `X` is a function:
  - the invocation is `X()`
- The imported entity can be renamed placing after the `import X` directive the `as Y`
  - That is:
    - `import <module or package> as Y`
    - `from <package> import <module or (sub)package or object> as Y`
    - `from <module> import <object> as Y`
- From such point on, `Y` must be used instead of `X`

Source: `https://chrisyeh96.github.io/2017/08/08/definitive-guide-python-imports.html`

Python as a functional language

- Already covered
- Lambda expressions
- Some kind of referential transparency
  - Immutable objects
  - Unchangeable (by default) global variables inside local scopes

About overloading of functions (1/3)

- In Python there is not a full overloading of functions as we know it in Java or C++
- If we define multiple times a function, only its latest definition will be used

```
>>> def f(a):
...     return a
>>> f(3)
3
>>> def f(a,b):
...     return (a,b)
>>> f(3,4)
(3, 4)
>>> f(3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: f() missing 1 required positional argument: 'b'
```

Source: https://www.geeksforgeeks.org/python-method-overloading/

About overloading of functions (2/3)

- The standard way to do it is via classes (see later)
- Default parameters may help
- They can be coupled with a defaulting to `None`
- Or we can use "decorators" (also explained later) but with care
- To use the required decorator we first need to install the suitable package `multipledispatch`

```
% pip3 install multipledispatch
Collecting multipledispatch
Downloading multipledispatch-1.0.0-py3-none-any.whl.metadata (3.8 kB)
Downloading multipledispatch-1.0.0-py3-none-any.whl (12 kB)
Installing collected packages: multipledispatch
Successfully installed multipledispatch-1.0.0
```

Source: `https://www.geeksforgeeks.org/python-method-overloading/`

About overloading of functions (3/3)

```
>>> from multipledispatch import dispatch
>>> @dispatch(int)
... def f(a):
...     return a
>>> @dispatch(int, int)
... def f(a,b):
...     return a+b
>>> f(3,4)
7
>>> f(3)
3
>>> def f(a,b):
...     return a-b
>>> f(3,4)
-1
>>> f(3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: f() missing 1 required positional argument: 'b'
```

Source: https://www.geeksforgeeks.org/python-method-overloading/

Functions are first class citizens

```
>>> def aFunction(a):
...     return a+1
>>> aFunction(3)
4
>>> x = aFunction
>>> x(3)
4
>>> l = [x, aFunction, 88, "bye"]
>>> l[0]
<function aFunction at 0x100ff0d60>
>>> l[1]
<function aFunction at 0x100ff0d60>
>>> l[2]
88
>>> l[3]
'bye'
>>> type(x)
<class 'function'>
```

Source: https://realpython.com/python-functional-programming/

Functions as parameters

- There are also predefined functions for this, such as `map`, `filter`, and `reduce`, which are used with the *map-reduce* pattern present in big data
- `map` returns an iterator

```
>>> def apply(f,x):
...     return f(x)
>>> def incr(x):
...     return x+1
...
>>> apply(incr,3)
4
>>> x = map(incr,[1,2,3,4])
>>> x
<map object at 0x100b9bf40>
>>> list(x)
[2, 3, 4, 5]
```

Source: `https://realpython.com/python-functional-programming/`

map plays a pivotal role

- It may take a different form using polymorphism
- Notice the polymorphic structure of the operator +
- map can be computed in constant time if there are enough processors, also taking advantage of a data-parallel approach

```
>>> def f(a,b,c):
...   return a+b+c
>>> f(1,2,3)
6
>>> f("a","b","c")
'abc'
>>> list(map(f,["a","b","c","d"],["e","f","g","h"],["i","j","k","l"]))
['aei', 'bfj', 'cgk', 'dhl']
>>>
```

Source: https://realpython.com/python-functional-programming/

`filter` selects elements

- The elements of a `filter` can be selected via the `filter`
- Inside `filter` the lambda functions are particularly handy
- `filter` returns an iterator
- Also `filter` can be computed in constant time if there are enough processors

```
>>> def f(a):
...     return a>10
>>> f(3)
False
>>> f(11)
True
>>> filter(f,[8,9,10,3,11,2,100,-11])
<filter object at 0x1006815d0>
>>> list(filter(f,[8,9,10,3,11,2,100,-11]))
[11, 100]
>>> list(filter(lambda x: x>10,[8,9,10,3,11,2,100,-11]))
[11, 100]
>>>
```

`reduce` synthesises elements

- The elements of an iterator can be folded in one with `reduce`
- They can be the result of a `map` and/or a `filter`
- The function for the reduction must have two elements, where the second must be of the type of the elements of the iterator
- `reduce` can be computed in log time if there are enough processors

```
>>> from functools import reduce
>>> def f(a,b):
...     return a+b
...
>>> reduce(f,[1,2,3,4,5])
15
>>> reduce(lambda x,y: x+y,[1,2,3,4,5])
15
>>> reduce(lambda x,y: x+y,map(lambda x: x**2,filter(lambda x: x %
    2!=0,[1,2,3,4,5])))
35
>>>
```

Lazy evaluation (1/3)

- Python has an eager evaluation of parameters passed to functions
- Python provides some form approximating lazy evaluation
- There are objects supplying a large stream of data one element at a time *on demand*, using the statement `yield` instead of the usual `return`
- This is very similar on how the elements of a list are accessed via iteration
- They can also be expanded in other data structures, like tuples
- Note that, once expanded, they are completely consumed, that is, lost

Source: `https://docs.python.org/3/howto/functional.html`

Lazy evaluation (2/3)

```
>>> l = [2,4,6,8,10,12]
>>> y = iter(l)
>>> y
<list_iterator object at 0x100681390>
>>> next(y)
2
>>> next(y)
4
>>> tuple(y)
(6, 8, 10, 12)
>>> tuple(y)
()
```

Source: `https://docs.python.org/3/howto/functional.html`

Lazy evaluation (3/3)

```
>>> def generateIntegers(N):
...     for i in range(N):
...         yield i
...
>>> x = generateIntegers(10)
>>> x
<generator object generateIntegers at 0x1002caf60>
>>> next(x)
0
>>> next(x)
1
>>> t = tuple(x)
>>> t
(2, 3, 4, 5, 6, 7, 8, 9)
>>> tuple(x)
()
>>> t
(2, 3, 4, 5, 6, 7, 8, 9)
```

Source: `https://docs.python.org/3/howto/functional.html`

List comprehension

- Python has very effective means to generate lists, as several functional languages; such means are collectively referred to with the term list comprehension
- Iterators are at the base of list comprehension
- List comprehension is heavily used in Python, as it is a very compact and expressive

```
>>> [x for x in range(10)]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> [x*x for x in (filter(lambda l: l%2==0,range(5,12)))]
[36, 64, 100]
>>> [x*x for x in range(5,12) if x%2==0]
[36, 64, 100]
>>> [x*x  if x%2==0 else -x*x for x in range(5,12)]
[-25, 36, -49, 64, -81, 100, -121]
```

Source: https://realpython.com/list-comprehension-python/ and https://www.w3schools.com/python/python_lists_comprehension.asp

Beyond List comprehension

- There are also *set comprehension* and *dictionary comprehension*
  - dictionary comprehension requires the pairs with :
- we could have nested or zip-ed comprehension
  - zip returns an iterator of tuples

```
>>> [x**2 for x in range(-3,3)]
[9, 4, 1, 0, 1, 4]
>>> {x**2 for x in range(-3,3)}
{0, 9, 4, 1}
>>> [(i,j) for i in range(3) for j in range(3)]
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]
>>> [(i,j) for (i,j) in zip(range(3),range(3))]
[(0, 0), (1, 1), (2, 2)]
>>> {i:j[::-1] for (i,j) in zip(range(1,4),["one","two","three"])}
{1: 'eno', 2: 'owt', 3: 'eerht'}
```

Source: https://www.freecodecamp.org/news/dictionary-comprehension-in-python-explained-with-examples/ and https://www.w3schools.com/python/ref_func_zip.asp

Python as an object oriented language (1/5)

- Python is an object oriented language (to a certain extent)
- Classes are defined with `class` and with the usual indentation
- The reference to the object itself is `self`
- Instance methods requires are identified by having as parameter `self`, which is then omitted when calling
- The initializer (a kind of constructor) is `__init__`
- Instance variables are *usually* defined in the constructor

```
class Animal:
    def __init__(self):
        self.name=""
    def setName(self,name):
        self.name=name
if __name__ == '__main__':
    animal = Animal()
    animal.setName("Pippo")
```

Python as an object oriented language (2/5)

- Class variables are defined globally in the class
- Class methods are defined with the `@classmethod` *decoration*, with a reference to the class, `cls`, and with the usual indentation

```
class Animal:
    numberOfAnimals = 0
    def __init__(self):
        Animal.numberOfAnimals += 1
        self.name=""
    def setName(self,name):
        self.name=name
    @classmethod
    def getNumberOfAnimals(cls):
        return cls.numberOfAnimals
if __name__ == '__main__':
    animal = Animal()
    animal.setName("Pippo")
    i = Animal.getNumberOfAnimals()
```

Python as an object oriented language (3/5)

- There are static methods non containing any reference to the class and defined with the `@staticmethod` decoration
  - Static methods exist only for naming reasons
- Protected methods and data start with the single underscore `_`
- Private methods and data start with the double underscore `__`
- However, both can be accessed outside the class with *name mangling*, that is, prefixing its name with `_ClassName`

```
class Animal:
    numberOfAnimals = 0
    def __init__(self, name):
        Animal.numberOfAnimals += 1
        self.name=name
        self.__private = 1
        self._protected = 1
        self.public = 1
```

Source: https://www.geeksforgeeks.org/private-methods-in-python/ and https://jellis18.github.io/post/2022-01-15-access-modifiers-python/

Python as an object oriented language (4/5)

```
    def identifyYourself(self):
        print("Hello! I am a generic animal and my name is " + self.name)

    def __privateChangeName(self, name):
        self.name = name

    @classmethod
    def getNumberOfAnimals(cls):
        return cls.numberOfAnimals

    @staticmethod
    def increment(i):
        return i+1
```

Python as an object oriented language (5/5)

```
if __name__ == '__main__':
    animal = Animal("Mistery")
    animal.identifyYourself()
    # animal.__privateChangeName("pippo")
    animal._Animal__privateChangeName("MoreMistery")
    i = Animal.getNumberOfAnimals()
    i = Animal.increment(i)
    animal._Animal_protected = i
    print(animal._Animal_protected)
    print(animal.name)
2
MoreMistery
```

Inheritance in Python

- Python supports single and multiple inheritance
- Every class without a superclass is implicitly derived from the class `object`
- `object` is then of type of `type`
- When a new object is created the initialized method `__init__` is called
- Here below a summary of a derived class

```
class Dog(Animal):

    def __init__(self, name):
        super().__init__(name)

    def identifyYourself(self):
        print("Hello! I am a dog and my name is " + self.name)
```

About `__init__` (1/2)

- Remember that `__init__` is an initializer, therefore, by default the creation of a derived class does not trigger the `__init__` of the base class
  - If an initialization from the superclass is needed, it needs to be explicitly called explicitly
- However, if there is no `__init__` in the derived class, the system will look for the closest one going up the hierarchy, since it is a virtual function

```
class Animal:
    def __init__(self):
        print("Creating an animal")

class Penguin(Animal):
    def __init__(self):
        print("Creating a penguin")
```

About __init__ (2/2)

```
class Parrot(Animal):
    def __init__(self):
        super().__init__()
        print("Creating a parrot")

class Dove(Animal):
        pass

if __name__ == "__main__":
    a = Animal()
    pe = Penguin()
    pa = Parrot()
    d = Dove()
```

- And therefore the output is:

```
Creating an animal
Creating a penguin
Creating an animal
Creating a parrot
Creating an animal
```

Operator overloading (1/5)

- A special kind of such member functions are those supporting operator overloading
- There is a specific mapping between names of functions and operator that is overloaded
- Such functions are “special functions” or “double underscore functions” or “magic methods”, such as `__add__()`

```
class Animal:

    def __add__(self,anotherAnimal):
        return Animal(self.name+anotherAnimal.name)

if __name__ == '__main__':
    animal = Animal("Mistery")
    anotherAnimal = Animal("BigMistery")
    thirdAnimal = animal + anotherAnimal
```

https://www.programiz.com/python-programming/operator-overloading/

Operator overloading (2/5)

- A list of the standard mapping of names and operator is presented in this and the following slide

+	A + B	__add__(self, other)
-	A – B	__sub__(self, other)
*	A * B	__mul__(self, other)
/	A / B	__truediv__(self, other)
//	A // B	__floordiv__(self, other)
%	A % B	__mod__(self, other)
**	A ** B	__pow__(self, other)
>>	A >> B	__rshift__(self, other)
<<	A << B	__lshift__(self, other)
&	A & B	__and__(self, other)
|	A | B	__or__(self, other)
^	A ^ B	__xor__(self, other)
<	A < B	__LT__(SELF, OTHER)
>	A > B	__GT__(SELF, OTHER)
<=	A <= B	__LE__(SELF, OTHER)

Source of the picture
`https://faun.pub/the-right-way-to-overload-methods-and-operators-in-python-2f93232af031/`

Operator overloading (3/5)

<=	A <= B	__LE__(SELF, OTHER)
>=	A >= B	__GE__(SELF, OTHER)
==	A == B	__EQ__(SELF, OTHER)
!=	A != B	__NE__(SELF, OTHER)
-=	A - = B	__ISUB__(SELF, OTHER)
+=	A += B	__IADD__(SELF, OTHER)
*=	A *= B	__IMUL__(SELF, OTHER)
/=	A /= B	__IDIV__(SELF, OTHER)
//=	A //= B	__IFLOORDIV__(SELF, OTHER)
%=	A %= B	__IMOD__(SELF, OTHER)
**=	A **= B	__IPOW__(SELF, OTHER)
>>=	A >>= B	__IRSHIFT__(SELF, OTHER)
<<=	A <<= B	__ILSHIFT__(SELF, OTHER)
&=	A &= B	__IAND__(SELF, OTHER)

Source of the picture
`https://faun.pub/the-right-way-to-overload-methods-and-operators-in-python-2f93232af031/`

Operator overloading (4/5)

- There are additional "double underscore" functions mimicking the structure of C++, such as
  - `__getitem__(self,index)` for subscripting in the rhs, that is the `x = anObject[index]`
  - `__setitem__(self,index)` for subscripting in the lhs, that is the `anObject[index] = x`
  - `__contains__(self,index)` for the `in` operator
  - `__repr__(self)` to convert an object in a string, used in `print` with the template pattern
  - `__iter__(self)` to generate an iterable, like `iter(anObject)` to use then, say, in a `for`
  - `__enter__(self)` and `__exit__(self,...)` to handle entering and exiting blocks

https://www.analyticsvidhya.com/blog/2021/08/explore-the-magic-methods-in-python/

Operator overloading (5/5)

- `__call__(self,whatever)` for *functional objects*, that is for managing structures like `anObject(whatever)`
  - It is also used to create objects as discussed later
- `__new__(...)` for allocating space for objects
- `__init__(...)` for initializing objects

`https://www.analyticsvidhya.com/blog/2021/08/explore-the-magic-methods-in-python/`

Understanding the `object` in Python (1/2)

- In Python an object has:
  - an identity
  - a value or a state, defined by the values of its attributes
  - a type
  - one or more bases, something similar to a superclass

```
>>> type(3)
<class 'int'>
>>> type(animal)
<class '__main__.Animal'>
>>> type(dog)
<class '__main__.Dog'>
>>> type(int)
<class 'type'>
>>> type(Animal)
<class 'type'>
>>> type(Dog)
<class 'type'>
```

Source: Shalabh Chaturvedi "Python Types and Objects"
https://www.eecg.toronto.edu/~jzhu/csc326/readings/metaclass-class-instance.pdf/

Understanding the `object` in Python (2/2)

- Also the `object` has a type
- And also the `type`!

```
>>> type(object)
<class 'type'>

>>> type(type)
<class 'type'>
```

Source: Shalabh Chaturvedi "Python Types and Objects"
https://www.eecg.toronto.edu/~jzhu/csc326/readings/metaclass-class-instance.pdf/

The notion of `object` (1/4)

- There is some duality to explore between type and base class, which requires some deepening
- First of all, we notice the reflection primitives, `type` and `dir`

```
>>> dir(3)
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '
    __delattr__', '__dir__', ...]
>>> dir(int)
['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__class__', '
    __delattr__', '__dir__', ...]
>>> dir(animal)
['_Animal__private', '_Animal__privateChangeName', '_Animal_protected', '
    __class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '
    __format__', '__ge__', '__getattribute__', '__getstate__', '__gt__', '
    __hash__', '__init__', '__init_subclass__', '__le__', '__lt__', '__module__
    ', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '
    __setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__', '
    _protected',  'getNumberOfAnimals', 'identifyYourself', 'increment', 'name'
    ,  'public']
```

Source: Shalabh Chaturvedi "Python Types and Objects"
https://www.eecg.toronto.edu/~jzhu/csc326/readings/metaclass-class-instance.pdf/

The notion of `object` (2/4)

```
>>> dir(dog)
['_Animal__private', '_Animal__privateChangeName', '__class__', '__delattr__', '
    __dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '
    __getattribute__', '__getstate__', '__gt__', '__hash__', '__init__', '
    __init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__',
    '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '
    __str__', '__subclasshook__', '__weakref__', '_protected',  '
    getNumberOfAnimals', 'identifyYourself', 'increment', 'name', '
    numberOfAnimals','public']
```

- To rationalize our understanding we need to explore the meaning of the attributes `__base__` and `__class__`
- We will use reflection again

Source: Shalabh Chaturvedi "Python Types and Objects"
https://www.eecg.toronto.edu/~jzhu/csc326/readings/metaclass-class-instance.pdf/

The notion of `object` (3/4)

```
>>> animal.__class__
<class '__main__.Animal'>
>>> type(animal.__class__)
<class 'type'>
>>> animal.__class__.__base__
<class 'object'>
>>>type(animal.__class__.__base__)
<class 'type'>

>>> dog.__class__
<class '__main__.Dog'>
>>> type(dog.__class__)
<class 'type'>
>>> dog.__class__.__base__
<class '__main__.Animal'>
>>>type(dog.__class__.__base__)
<class 'type'>
```

Source: Shalabh Chaturvedi "Python Types and Objects"
https://www.eecg.toronto.edu/~jzhu/csc326/readings/metaclass-class-instance.pdf/

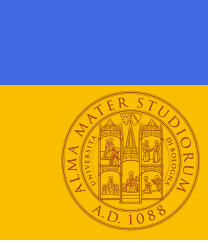
The notion of `object` (4/4)

- Now we can explore the objects `object` and `type`

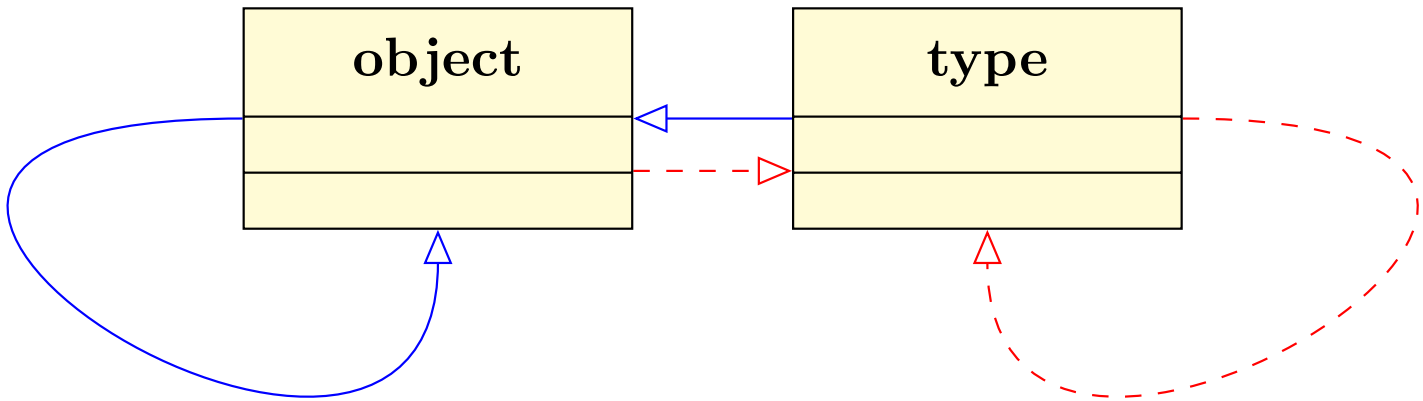
```
>>> object.__class__
<class 'type'>
>>> type(object.__class__)
<class 'type'>
>>> object.__class__.__base__
<class 'object'>
>>> type(object.__class__.__base__)
<class 'type'>

>>> type.__class__
<class 'type'>
>>> type(type.__class__)
<class 'type'>
>>> type.__class__.__base__
<class 'object'>
>>> type(type.__class__.__base__)
<class 'type'>
```

Source: Shalabh Chaturvedi "Python Types and Objects"
https://www.eecg.toronto.edu/~jzhu/csc326/readings/metaclass-class-instance.pdf/



A view of `object` and `type` in UML



Revising our objects (1/5)

- *(Class)* `Animal`
  - identity (`Animal.__name__`) : `Animal`
  - a type (`Animal.__class__`) : `type`
  - a base (`Animal.__base__`) : `object`
- *(Object)* `animal`
  - there is no attribute `animal.__name__`
  - a type (`Animal.__class__`) : `__main__.Animal`
  - there is no attribute `animal.__base__`

Revising our objects (2/5)

- *(Class)* `Dog`
  - identity (`Dog.__name__`) : `Dog`
  - a type (`Dog.__class__`) : `type`
  - a base (`Dog.__base__`) : `__main__.Animal`
- *(Object)* `dog`
  - there is no attribute `dog.__name__`
  - a type (`dog.__class__`) : `__main__.Dog`
  - there is no attribute `dog.__base__`

Revising our objects (3/5)

- *(Object)* `object`
  - identity (`object.__name__`) : `object`
  - a type (`object.__class__`) : `type`
  - a base (`object.__base__`) : `None`
- *(Class)* `object`
  - identity (`object.__class__.__name__`) : `type`
  - a type (`object.__class__.__class__`) : `type`
  - a base (`object.__class__.__base__`) : `None`

Revising our objects (4/5)

- *(Base class of object)* aka `object.__base__` `None`
  - there is no attribute `object.__base__.__name__`
  - a type (`object.__base__.__class__`) : `NoneType`
  - there is no attribute `object.__base__.__base__`
- *(Type of the base class of object)* aka `None.__base__` aka `object.__base__.__class__`: `NoneType`
  - identity (`object.__base__.__class__.__name__`) : `NoneType`
  - type (`object.__base__.__class__.__class__`) : `type`
  - there is no attribute `object.__base__.__class__.__base__`

Revising our objects (5/5)

- *(Object)* `type`
  - identity (`type.__name__`) : `type`
  - a type (`type.__class__`) : `type`
  - a base (`type.__base__`) : `object`
- *(Class of)* `type`
  - identity (`object.__class__.__name__`) : `object`
  - a type (`object.__class__.__class__`) : `type`
  - a base (`object.__class__.__base__`) : `type`
  - **It is the `object`, but with some inconsistency!!**

Putting the pieces together

- We have the structure of the Abstract Factory
- Everything is (derived of) an `object`
- Every object has a `type`
- A `type` is an object and derived (also indirectly) from `object`
- The `type` is identified by the `__base__` attribute
- The base class is identified by the `__class__` attribute
- A metaclass is a class whose instances are still classes
- Metaclasses play an important role in the creation/instantiation process

Source: Shalabh Chaturvedi "Python Types and Objects"
https://www.eecg.toronto.edu/~jzhu/csc326/readings/metaclass-class-instance.pdf/

Instantiation of new objects

- The reference for creating new objects is the clone design pattern
- New objects can be created by subclassing

```
class Dog(Animal):

    def __init__(self, name):
        super().__init__(name)
```

- A new object of type `type` has been instantiated with base class `object` (the full specs in slide 124)
- New objects can be created by applying the operator `()` to an object of type `type`

```
dog = Dog()
o = object()
animal = Animal()
```

Source: Shalabh Chaturvedi "Python Types and Objects"
https://www.eecg.toronto.edu/~jzhu/csc326/readings/metaclass-class-instance.pdf/

Instantiation of new types (1/2)

- Types can be created on the fly
- Extreme care is required

```
>>> Cat = type("Cat",(Animal,),{"nickname":"Prr", "weight":10})
>>> c = Cat("Garfield")
>>> print(c)
<__main__.Cat object at 0x11148fa90>
>>> print(c.nickname)
Prr
>>> print(c.weight)
10
>>> print(c.name)
Garfield
>>> print(c.somethingElse)
    print(c.somethingElse)
          ^^^^^^^^^^^^^^
AttributeError: 'Cat' object has no attribute 'somethingElse'
>>> c.somethingElse=2
>>> print(c.somethingElse)
2
```

Source: https://realphysics.info/Theory%20of%20Python/classes.html/

Instantiation of new types (2/2)

- Types can be returned from functions

```
    def generateCongruent(something):
        return type(something)
    t = generateCongruent(c)
    x = t("Garfield2, the revenge")
    type(x)
    Cat   Garfield2, the revenge
    type(x.something)
    ^^^^^^^^^^^
AttributeError: 'Cat' object has no attribute 'something'
```

Source: https://realphysics.info/Theory%20of%20Python/classes.html/

The creation process (1/2)

- The creation of an object takes two (plus one) steps:
  - `__new__()`, in principle to allocate memory
  - `__init__()`, in principle to initialize data
- The original object is taken as a reference
- It is also possible to modify the creation of objects using metaclasses
  - Using metaclasses it is possible to alter the creation process of objects
  - In this case, a rigid protocol is employed, using the template pattern, again, based on overriding / late binding

Source: Shalabh Chaturvedi "Python Types and Objects"
https://www.eecg.toronto.edu/~jzhu/csc326/readings/metaclass-class-instance.pdf/,
https://www.datacamp.com/tutorial/python-metaclasses, and
https://stackoverflow.com/questions/56514586/arguments-of-new-and-init-for-metaclasses

The creation process (2/2)

- Overall we see three methods clearly involved in the protocol
  - `__call__()`, the orchestrator, called every time an object type is invoked to create a new object, like in

```
dog = Dog()
```

  - `__new__()`, called to allocate memory, as mentioned
  - `__init__()`, called to initialize the data, as mentioned
- `__call__()` calls the other two, therefore imposing a structure on the number and types of parameters
- this is why we may see typing error if they are not properly structured

Source: Shalabh Chaturvedi "Python Types and Objects"
https://www.eecg.toronto.edu/~jzhu/csc326/readings/metaclass-class-instance.pdf/,
https://www.datacamp.com/tutorial/python-metaclasses, and
https://stackoverflow.com/questions/56514586/arguments-of-new-and-init-for-metaclasses

Inside the creation (1/5)

- The creation process involves metaclasses
- `type` is acting as the "metametaclass"
- `*` and `**` allow arbitrary number of parameters:
  - `*` implies that the argument (here, `args`) is a tuple
  - `**` implies that the argument (here, `kw`) is a dictonary
  - Here we see the equivalent in Python of the real C code (taken from the reference below)

```
class type:
    def __call__(cls, *args, **kw):
        instance = cls.__new__(cls, *args, **kw)
        # __new__ is actually a static method -
        # cls has to be passed explicitly
        if isinstance(instance, cls):
            instance.__init__(*args, **kw)
        return instance
```

Source: https://www.datacamp.com/tutorial/python-metaclasses, and https://stackoverflow.com/questions/56514586/arguments-of-new-and-init-for-metaclasses

Inside the creation (2/5)

- Substantially `__new__()` and `__init__()` ought to be redefined
- Let's see an implementation of the singleton pattern

```
class Singleton:
    singleObject = None
    @staticmethod
    def __new__(cls):
        if Singleton.singleObject is not None:
            return Singleton.singleObject
        else:
            Singleton.singleObject = super().__new__(cls)
            return Singleton.singleObject

    def __init__(self):
        self.aValue = 1

    def increment(self):
        self.aValue += 1
```

Inside the creation (3/5)

- What is the output?

```
>>> if __name__ == "__main__":
>>>     x = Singleton()
>>>     print(x.aValue)
>>>     x.increment()
>>>     print(x.aValue)
>>>     y = Singleton()
>>>     print(x.aValue)
>>>     print(y.aValue)
>>>     print(x==y)
```

Inside the creation (4/5)

```
>>> if __name__ == "__main__":
>>>     x = Singleton()
>>>     print(x.aValue)
1
>>>     x.increment()
>>>     print(x.aValue)
2
>>>     y = Singleton()
>>>     print(x.aValue)
1
>>>     print(y.aValue)
1
>>>     print(x==y)
True
```

- `__init__()` is still called!!
- We need some refinement
  - Remember that for `type` we cannot modify the sequence of calling first `__init__()` and then `__new__()`
    - it is built inside the language

Inside the creation (5/5)

```
    def __init__(self):
        if Singleton.numberOfItems == 0:
            Singleton.numberOfItems = 1
            self.aValue = 1

>>> if __name__ == "__main__":
>>>     x = Singleton()
>>>     print(x.aValue)
1
>>>     x.increment()
>>>     print(x.aValue)
2
>>>     y = Singleton()
>>>     print(x.aValue)
2
>>>     print(y.aValue)
2
>>>     print(x==y)
True
```

Metaclasses (1/9)

- Remember that in Python everything is an object, including classes
- As mentioned,
  - a metaclass is a class whose instances are still classes
  - the creation process involves metaclasses
  - `type` is acting as the "metametaclass"
- The instruction

```
class Animal:
    def __init__(self):
        print("Creating an animal")
```

results in the creation (instantiation) of:

- a class `Animal` implicitly derived from class `object`
- an object `Animal` instance of the class `type`

Source: `https://www.datacamp.com/tutorial/python-metaclasses`

Metaclasses (2/9)

- Moreover, with this code

```
animal = Animal()
```

  the object `Animal` creates an instance of class `Animal` and returns its value to `animal`
  - This is clearly an use of the method `__call__()`
- So the question is whether it is possible for the class `Animal` to produce also other classes
- This is the core of metaclasses

Source: `https://www.datacamp.com/tutorial/python-metaclasses`

Metaclasses (3/9)

- We start with defining a subclass of class `type`

```
class AnimalType(type):
    pass
```

- As usual here we create
  - a class `AnimalType` explicitly derived from class `type`
  - an object `AnimalType` instance of the class `type`

Source: `https://www.datacamp.com/tutorial/python-metaclasses`

Metaclasses (4/9)

- Then we use such class to proceed

```
class Animal(metaclass=AnimalType):
    pass
```

- And here we create
  - a class `Animal` with:
    - meta-class class `AnimalType`, meaning that its associated object is instance of class `AnimalType`, and
    - implicitly derived from class `object`
  - an object `Animal` instance of the class `AnimalType`, as mentioned above

Source: `https://www.datacamp.com/tutorial/python-metaclasses`

Metaclasses (5/9)

- Metaclassing allows the manipulation of the class as a whole, especially its creation process
- We now consider an example

```
class AnimalType(type):
    def __init__(cls, name, bases, dct):
        print("__init__ of AnimalType")
    def __call__(self, *args, **kwargs):
        print("__call__ of AnimalType")
        return super().__call__(self, *args, **kwargs)
    def __new__(cls, name, *args, **kwargs):
        print("__new__ of AnimalType")
        return super().__new__(cls, name, *args, **kwargs)
 class Animal(metaclass=AnimalType):
    def __init__(self,*args):
        print("__init__ of Animal")
    def __new__(cls, *args, **kwargs):
        print("__new__ of Animal")
        return super().__new__(cls)
```

Source: `https://www.datacamp.com/tutorial/python-metaclasses`

Metaclasses (6/9)

```
__new__ of AnimalType # To create the object AnimalType at the beginning of the
    module
__init__ of AnimalType # To initialize the object AnimalType
>>> if __name__ == '__main__':
>>>     print("Now creating an Animal")
Now creating an Animal
>>>     a = Animal()
__call__ of AnimalType # This is the result of the call Animal(
__new__ of Animal # Called by AnimalType.__call__
__init__ of Animal # Called by AnimalType.__call__
```

- The `__call__()` of the metaclass (in our example class `AnimalType`) orchestrates the overall creation process of the objects of class `Animal`
  - It is an application of the *Template Method* design pattern
- This allows a fine-grained control of the creation process

Source: `https://www.datacamp.com/tutorial/python-metaclasses`

Metaclasses (7/9)

- The `__call__()` of the metaclass calls of the target class `__new__()` and `__init__()`
- Therefore, the metaclass has its own:
  - `__call__(cls, *args, **kwargs)`, called when an instance of the metaclass is called, for instance when creating an object of the class, like in `a = Animal()`
    - `cls` is the reference to the metaclass
    - `args` is the list of positional arguments
    - `kwargs` is the list of the keyword arguments

Source:
https://www.datasciencecentral.com/understanding-the-complexity-of-metaclasses-and-their-practical-2/

Metaclasses (8/9)

- . . . therefore, the metaclass has its own:
  - `__new__(mcs, name, bases, namespace)`, where
    - `mcs` is the reference to the metaclass
    - `name` is the name to the metaclass
    - `bases` is the list of the superclasses, which will become the `__base__` attribute of the new class
    - `namespace` is the namespace in the form of a dictionary, which will become the `__dict__` attribute of the new class

Source:
`https://www.datasciencecentral.com/understanding-the-complexity-of-metaclasses-and-their-practical-2/`

Metaclasses (9/9)

- ... therefore, the metaclass has its own:
  - `__init__(cls, name, bases, namespace, **kwargs)`
    - `mcs` is the reference to the metaclass
    - `name` is the name to the metaclass
    - `bases` is the list of the superclasses, which will become the `__base__` attribute of the new class
    - `namespace`is the namespace in the form of a dictionary, which will become the `__dict__` attribute of the new class
    - `kwargs` is the list of the keyword arguments
- There is also a `__prepare__()` method, called as the first method to prepare the namespace

Source:
https://www.datasciencecentral.com/understanding-the-complexity-of-metaclasses-and-their-practical-2/

Singleton with metaclasses (1/4)

- We can use it for an alternative implementation of the Singleton pattern

```
class AnimalType(type):
    singleAnimal=None
    def __init__(cls, name, bases, dct):
        print("__init__ of AnimalType")

    def __call__(cls, *args, **kwargs):
        print("__call__ of AnimalType")
        if AnimalType.singleAnimal is None:
            print("Creating the unique animal")
            AnimalType.singleAnimal = cls.__new__(cls, *args, **kwargs)
            AnimalType.singleAnimal.__init__(*args, **kwargs)
        else:
            print("The unique animal already exists")
        return AnimalType.singleAnimal

    def __new__(cls, name, *args, **kwargs):
        print("__new__ of AnimalType")
        return super().__new__(cls, *args, **kwargs)
```

Singleton with metaclasses (2/4)

```
class Animal(metaclass=AnimalType):
    def __init__(self,*args):
        print("__init__ of Animal")
        print(f'self = {self}; args = {args}')
        self.weight=10
        print(f'self.weight = {self.weight}; args = {args}')

    def __new__(cls, *args, **kwargs):
        print("__new__ of Animal")
        if AnimalType.singleAnimal is  None:
            print("Creating the unique singleton")
            AnimalType.singleAnimal = super().__new__(cls, *args, **kwargs)
        return AnimalType.singleAnimal
```

Singleton with metaclasses (3/4)

- Now we start the execution

```
__new__ of AnimalType # The object AnimalType is created
__init__ of AnimalType
>>> if __name__ == '__main__':
>>>     print("Now creating an Animal")
Now creating an Animal
>>>     print(type(AnimalType))
<class 'type'>
>>>     print(type(Animal))
<class '__main__.AnimalType'>
>>>     a = Animal() # This is a call to the singleton;
>>>.    # there is no need of a method called getSingleton
__call__ of AnimalType
Creating the unique animal
__new__ of Animal
Creating the unique singleton
__init__ of Animal
```

Singleton with metaclasses (4/4)

```
>>>     print(a)
<__main__.Animal object at 0x106f24ed0>
>>>     print(a.weight)
10
>>>     a.weight = 20
>>>     print(a.weight)
20
>>>     b = Animal() # Having overloaded AnimalType.__call__(),
>>>.    # no new object is created but the singleton is returned
The unique animal already exists
>>>     print(b)
<__main__.Animal object at 0x106f24ed0>
>>>     print(a==b)
True
>>>     b.weight = 300
>>>     print(a.weight)
300
```

Generalization of Singleton (1/2)

- It is possible to create a general meta class to use when we need a singleton of any class

```
class Singleton(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls]=super(type(cls), cls).__call__(*args, **kwargs)
        return cls._instances[cls]
```

- Here we let `__call__` invoke the `__call__` of the superclass only if no instances of `cls` have already been created

Source: `https://itnext.io/deciding-the-best-singleton-approach-in-python-65c61e90cdc4`

Generalization of Singleton (2/2)

- Now suppose that we want to create a specialization of a Database class allowing only an instance of it

```
class Database():
    def __init__(self, url):
        self.url = url

    def connect(self):
        if 'https' not in self.url:
            raise ValueError("invalid url: it must be encrypted")
        return True

class DatabaseSingleton(Database, metaclass=Singleton):
    pass
```

- Specifying as metaclass `Singleton` we have achieved our goal

Source: `https://itnext.io/deciding-the-best-singleton-approach-in-python-65c61e90cdc4`

Type Hints (1/8)

Unlike languages like C/C++, Python uses **dynamic typing**:

- Variables, parameters, and return values can be of any type
- Their type can **change at runtime**

This makes coding easier, but can lead to runtime errors.

- **Type hints** add **optional static typing** to help catch errors earlier, combining the benefits of both static and dynamic typing.

Source: `https://www.pythontutorial.net/python-basics/python-type-hints/`

Type Hints (2/8)

- This example shows a function that receives a string as input and returns a string as output

```
def say_hi(name):
    return f'Hi {name}'
```

- Here is the syntax for adding type hints to the name parameter and the return value of the `say_hi()` function:

```
def say_hi(name: str) -> str:
    return f'Hi {name}'

>>> greeting = say_hi('John')
>>> print(greeting)
Hi John
```

- In this new syntax, the `name` parameter and the return value of the `say_hi()` function have the str type

Source: `https://www.pythontutorial.net/python-basics/python-type-hints/`

Type Hints (3/8)

- In addition to the str type, you can use other built-in types like int, float, bool, and bytes for type hints
- The Python interpreter **ignores** type hints
  - For example, if you pass a number to the `say_hi()` function, it will still run without any warning or error
- To verify type hints, you need to use a static type checker such as mypy

Source: `https://www.pythontutorial.net/python-basics/python-type-hints/`

Type Hints (4/8)

- Python doesn't have an official static type checker tool, the most popular third-party tool is **Mypy**

```
>>> greeting = say_hi(123)
>>> print(greeting)
Hi 123

>>> pip instal mypy      # installing mypy
>>> mypy app.py          # checking the type before running the program
app.py:5: error: Argument 1 to "say_hi" has incompatible type "int"; expected "
    str"
Found 1 error in 1 file (checked 1 source file)
```

- The error shows that an int was passed to `say_hi()`, but a str was expected

Source: `https://www.pythontutorial.net/python-basics/python-type-hints/`

Type Hints (5/8)

- You can add a type hint when defining a variable, although it's often unnecessary since static type checkers usually infer the type from the assigned value:

```
>>> name: str = 'Hello' #equivalent to name = 'Hello'
>>> name = 100   # assigning a value that is not a string to the name variable

app.py:2: error: Incompatible types in assignment (expression has type "int",
    variable has type "str")
Found 1 error in 1 file (checked 1 source file)
```

Source: https://www.pythontutorial.net/python-basics/python-type-hints/

Type Hints (6/8)

- Numbers can be either integers or floats. You can use **Union** to create a type hint that accepts both int and float

```
from typing import Union
def add(x: Union[int, float], y: Union[int, float]) -> Union[int, float]:
    return x + y

#Starting from Python 3.10, you can use the X | Y syntax to create a union type:
def add(x: int | float, y: int | float) -> int | float:
    return x + y
```

- Python allows you to create a type alias and use it for type hinting:

```
number = Union[int, float]
def add(x: number, y: number) -> number:
    return x + y
```

Source: https://www.pythontutorial.net/python-basics/python-type-hints/

Type Hints (7/8)

- You can use type hints also in collections
- You can use type aliases from the typing module to specify the types of values inside lists, dictionaries, and sets:

Type Alias	**Built-in Type**
`List`	`list`
`Tuple`	`tuple`
`Dict`	`dict`
`Set`	`set`
`Frozenset`	`frozenset`
`Sequence`	For `list`, `tuple`, and other sequence types
`Mapping`	For `dict` and other mapping types
`ByteString`	`bytes`, `bytearray`, and `memoryview`

Source: `https://www.pythontutorial.net/python-basics/python-type-hints/`

Type Hints (8/8)

- Here are some examples:

```python
from typing import List, Dict, Set

# List of integers
nums: List[int] = [1, 2, 3]

# Dictionary with string keys and int values
ages: Dict[str, int] = {"Alice": 25, "Bob": 30}

# Set of strings
tags: Set[str] = {"python", "typing"}
```

- If a function does not explicitly return a value, you can use **None** as the return type hint

```python
def log(message: str) -> None:
    print(message)
```

Source: `https://www.pythontutorial.net/python-basics/python-type-hints/`

Reflection and Introspection (1/8)

- **Introspection** is the ability to discover information about an object at runtime
- **Reflection** extends introspection by allowing the modification of objects at runtime

- The most basic info you can get at runtime is an object's **type**
- Use the `type()` function to get an object's type:

```
>>> type(1)
<class 'int'>
>>> type(1.0)
<class 'float'>
>>> type(int)
<class 'type'>
```

- The return value of `type()` is a **type object**, telling us the class of the argument

Source: `https://betterprogramming.pub/python-reflection-and-introspection-97b348be54d8`

Reflection and Introspection (2/8)

- Use `isinstance()` to check if an object is an instance of a class:

```
>>> isinstance(1, int)
True
>>> isinstance(int, type)
True
```

- You can also compare types with `type()`:

```
>>> type(1) is int
True
>>> type(int) is type
True
```

- Unlike `type()`, `isinstance()` considers subclassing

Source: `https://betterprogramming.pub/python-reflection-and-introspection-97b348be54d8`

Reflection and Introspection (3/8)

- It is useful to combine `type()` and `isinstance()`:

```
if isinstance(obj, A):
    # do something for all children of A

    if type(obj) is B:
        # do something specifically for instances of B
```

- This pattern is useful to distinguish subclasses from exact class matches

Source: `https://betterprogramming.pub/python-reflection-and-introspection-97b348be54d8`

Reflection and Introspection (4/8)

- Other useful introspection functions:
  - `hasattr(obj, 'a')` — checks if `obj` has attribute `'a'`
  - `id(obj)` — returns the unique ID of the object
  - `dir(obj)` — lists attributes and methods of `obj`
  - `vars(obj)` — returns instance variables in a dictionary
  - `callable(obj)` — returns `True` if `obj` can be called like a function

Source: `https://betterprogramming.pub/python-reflection-and-introspection-97b348be54d8`

Reflection and Introspection (5/8)

- In Python it is also possible to modify or create objects and classes dynamically
  - You can add attributes to a class or object at runtime:

```
>>> class A:
...     pass    # Define an empty class A
>>> A.x = 1     # Add a class attribute 'x' with value 1
>>> a = A()     # Create an instance 'a' of class A
>>> a.y = 2     # Dynamically add an instance attribute 'y' with value 2
>>> a.y         # Access the instance attribute 'y' -> returns 2
2
>>> a.x         # Access the class attribute 'x' via instance 'a' -> returns 1
1
```

Source: https://betterprogramming.pub/python-reflection-and-introspection-97b348be54d8

Reflection and Introspection (6/8)

- Since methods are just special attributes, you can add methods dynamically as well:

```
>>> def init(self):                    # Define a function to be used as __init__
...     self.x = 1
>>> class A:
...     pass                           # Define an empty class A
>>> setattr(A, '__init__', init)       # Assign __init__ as the init method of class A
>>> a = A()                            # Create an instance of A, which calls init()
>>> a.x                                # Access the instance attribute x set by init()
1
```

- Here, `setattr` sets the `__init__` attribute of class `A` dynamically

Source: `https://betterprogramming.pub/python-reflection-and-introspection-97b348be54d8`

Reflection and Introspection (7/8)

- You can even modify a function's code by assigning to its `__code__` attribute:

```
>>> def test():
...     print("Test")
>>> test()
Test
>>> test.__code__ = (lambda: print("Hello")).__code__  # Replace test function's
                                                       code with lambda's code
>>> test()                  # Calls the modified test function, now prints "Hello"
Hello
```

Source: `https://betterprogramming.pub/python-reflection-and-introspection-97b348be54d8`

Reflection and Introspection (8/8)

Creating classes at runtime with `type()`:

- `type(name, bases, dict)` creates a new class dynamically:
  - `name`: class name (string)
  - `bases`: tuple of base classes
  - `dict`: dictionary of attributes and methods

```
>>> A = type('A', (), {'x': 1}) # Dynamically create class A with attribute x = 1
>>> a = A()                     # Instantiate object a of class A
>>> a.x                         # Access attribute x of object a
1
```

Source: https://betterprogramming.pub/python-reflection-and-introspection-97b348be54d8

Decorators (1/24)

- A **decorator** is a higher-order function that dynamically alters the behavior of functions or classes by wrapping them in another callable.
- Common use cases: logging, authentication, memorization

```
def decorator(func):          # Define a decorator
    def wrapper():            # wrapper adds behavior before and after 'func'
        print("Before calling the function.")
        func()                                  # Call the original function
        print("After calling the function.")
    return wrapper

@decorator                    # Apply the decorator to the greet function
def greet():
    print("Hello, World!")    # Original function behavior

>>> greet()                   # Call the decorated function
Before calling the function.
Hello, World!
After calling the function.
```

Source: https://www.geeksforgeeks.org/decorators-in-python/

Decorators (2/24)

- **Higher-order functions** are functions that take or return a function: decorators are an example
- In Python, functions are **first-class objects** that can be:
  - **Assigned to variables**
  - **Passed** as arguments
  - **Returned** from functions
  - **Stored** in data structures

```
def greet(n):                # Assigning a function to a variable
return f"Hello, {n}!"

say_hi = greet               # Assign the greet function to say_hi
print(say_hi("Alice"))       # Output: Hello, Alice!
```

Source: https://www.geeksforgeeks.org/decorators-in-python/

Decorators (3/24)

- Another example:

```
def make_mult(f):
    def mult(x):
    return x * f
return mult

dbl = make_mult(2)
print(dbl(5))        # Output: 10
```

- `make_mult` returns a function that multiplies input by `f`
- This mechanism is fundamental for decorators

Decorators (4/24)

- Syntax of decorator parameters:

```
def decorator_name(func):   # 'func' represents the function being decorated
    def wrapper(*args, **kwargs):
        # Code before
        result = func(*args, **kwargs)
        # Code after
    return result
return wrapper

@decorator_name
def function_to_decorate():
pass
```

- *args captures positional arguments as a tuple
- **kwargs captures keyword arguments as a dictionary

Source: https://www.geeksforgeeks.org/decorators-in-python/

Decorators (5/24)

- Decorators work because functions are **first-class** and support **higher-order** behavior.
- Core mechanism:
  - Take a function
  - Return a new function
  - The original function is replaced

```
@decorator

def func():
pass

# Equivalent to:

func = decorator(func)
```

Source: `https://www.geeksforgeeks.org/decorators-in-python/`

Decorators (6/24)

- There are 3 main types of decorators:
  - **Function** decorators (as seen in the first example)
  - **Method** decorators: used to decorate class methods
  - **Class** decorators: used to modify or enhance the behavior of a class

Decorators (7/24)

Example of a method decorator

```
def method_decorator(func):                    # func is the method being decorated
    def wrapper(self, *args, **kwargs):
        # self is required because this is an instance method
        # *args and **kwargs allow the method to accept any arguments
        print("Before method execution")
        res = func(self, *args, **kwargs)      # Call the original method
        print("After method execution")
        return res
    return wrapper                             # Return the new wrapped function
class MyClass:
    @method_decorator              # Apply the decorator to the say_hello method
    def say_hello(self):
        print("Hello!")
obj = MyClass()                    # Create an instance of MyClass
obj.say_hello()                    # Call the method
# Output:
# Before method execution
# Hello!
# After method execution
```

Source: https://www.geeksforgeeks.org/decorators-in-python/

Decorators (8/24)

Example of a class decorator

```
def add_class_name(cls):                    # cls is the class being decorated
    cls.class_name = cls.__name__           # Add a new attribute 'class_name'
    return cls                              # Return the modified class

@add_class_name         # Apply the decorator to the Person class
class Person:
    pass

print(Person.class_name)    # Access the added class attribute

# Output:
# Person
```

Source: https://www.geeksforgeeks.org/decorators-in-python/

Decorators (9/24)

Some common built-in decorators in Python:

- `@staticmethod`: method does not depend on instance

```
class MathOperations:
    @staticmethod
    def add(x, y):
        return x + y
print(MathOperations.add(5, 3))  # Output: 8
```

- `@classmethod`: method works with class (uses `cls`)

```
class Employee:
    raise_amount = 1.05
    @classmethod
    def set_raise_amount(cls, amount):
        cls.raise_amount = amount

Employee.set_raise_amount(1.10)
print(Employee.raise_amount)  # Output: 1.1
```

Source: `https://www.geeksforgeeks.org/decorators-in-python/`

Decorators (10/24)

- `@staticmethod` wraps a function to be called without instance or class

```
class staticmethod:
    def __init__(self, func):
        self.func = func
    def __get__(self, instance, owner):
        return self.func
```

- `@classmethod` passes the class as the first argument to the method

```
class classmethod:
    def __init__(self, func):
        self.func = func
    def __get__(self, instance, owner):
        def new_func(*args, **kwargs):
            return self.func(owner, *args, **kwargs)
        return new_func
```

Source: `https://www.geeksforgeeks.org/decorators-in-python/`

Decorators (11/24)

Instance method (no decorator): requires an instance to be called

```
class Dog:
    def speak(self):
        return "Woof"

d = Dog()
print(d.speak())        # OK: Output "Woof"
print(Dog.speak())      # Error: missing 1 required positional argument: 'self'
```

- Requires an instance: `self` is passed automatically.
- Calling from the class directly causes an error.

Source: `https://www.geeksforgeeks.org/decorators-in-python/`

Decorators (12/24)

Static & Class methods: callable without instance

```
class Dog:
    @staticmethod
    def bark():
        return "Woof"

    @classmethod
    def species(cls):
        return cls.__name__

print(Dog.bark())       # OK: Output "Woof"
print(Dog.species())    # OK: Output "Dog"
```

- `@staticmethod`: no self or cls; behaves like a regular function.
- `@classmethod`: receives the class (`cls`) as first argument.

Source: `https://www.geeksforgeeks.org/decorators-in-python/`

Decorators (13/24)

- `@property`: getter/setter for attribute access control

```
class Circle:
    def __init__(self, radius):
        self._radius = radius           # Initialize a "private" attribute
    @property
    def radius(self):
        return self._radius             # Getter for radius
    @radius.setter
    def radius(self, value):
        if value >= 0:
            self._radius = value        # Setter with validation
        else:
            raise ValueError("Radius cannot be negative")
    @property
    def area(self):
        return 3.14159 * self._radius ** 2  # Read-only property to compute area
c = Circle(5)
print(c.radius)     # Output: 5 (access via getter)
print(c.area)       # Output: 78.53975 (computed via property)
```

Source: `https://www.geeksforgeeks.org/decorators-in-python/`

Decorators (14/24)

- Why `@property` is better than direct access

Direct attribute access:

```
class Circle:
    def __init__(self, radius):
        self.radius = radius  # No
     control

c = Circle(-10)  # No error!
print(c.radius)  # -10
```

No validation, easy to misuse.

Using `@property`:

```
class Circle:
    def __init__(self, radius):
        self.radius = radius
    @property
    def radius(self):
        return self._radius
    @radius.setter
    def radius(self, value):
        if value >= 0:
            self._radius = value
        else:
            raise ValueError("Radius
     cannot be negative")

c = Circle(5)
c.radius = -10  # Raises error!
```

Validates input, encapsulates data.

Decorators (15/24)

- `@property` turns a method into a **property** that can be accessed like an attribute
- Allows defining **getter** methods without explicitly calling them
- Encapsulates attribute access logic while keeping simple attribute-like syntax
- You can also add `@<property>.setter` and `@<property>.deleter` to manage assignment and deletion

Source: `https://realpython.com/python-property/`

Decorators (16/24)

Applying multiple decorators is called **decorator chaining**

```
def decor1(func):
    def inner():
        x = func()
        return x * x
    return inner

def decor(func):
    def inner():
        x = func()
        return 2 * x
    return inner

@decor1
@decor
def num():
    return 10

print(num())  # Output: 400
```

```
@decor
@decor1
def num2():
    return 10

print(num2())  # Output: 200
```

Note: Order matters!

`@decor1 @decor` → square then double

`@decor @decor1` → double then square

Source: https://www.geeksforgeeks.org/decorators-in-python/

Decorators (17/24)

- ABCs define a common interface for subclasses
- Use abstract methods to enforce mandatory implementations
- Ensure consistent structure among subclasses
- Provided by the `abc` module

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass
```

- `Animal` is abstract and cannot be instantiated
- `sound` method must be implemented by subclasses

Source: `https://www.geeksforgeeks.org/abstract-classes-in-python/`

Decorators (18/24)

- A subclass remains **abstract** if it does not implement all methods marked with `@abstractmethod`
- Such a class cannot be instantiated; Python raises a `TypeError`
- When the subclass overrides all abstract methods, it becomes **concrete** and instantiable
- A subclass can also declare new abstract methods, making it abstract again

```
class Dog(Animal):  # still abstract
    pass

class Cat(Animal):  # concrete class
    def sound(self):
        print("Meow")
```

Source: `https://www.geeksforgeeks.org/abstract-classes-in-python/`

Decorators (19/24)

- Example of a concrete subclass implementing the abstract method

```
class Dog(Animal):
    def sound(self):
        return "Bark"

dog = Dog()
print(dog.sound())  # Output: Bark
```

- `Dog` can be instantiated because it implements `sound()`
- Missing implementation raises an error

Source: `https://www.geeksforgeeks.org/abstract-classes-in-python/`

Decorators (20/24)

- ABCs can have concrete methods with full implementation
- Subclasses inherit these methods without needing to override

```
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

    def move(self):
        return "Moving"
```

- `move()` is a concrete method usable as-is
- `make_sound()` remains abstract and must be implemented

Source: `https://www.geeksforgeeks.org/abstract-classes-in-python/`

Decorators (21/24)

- Abstract properties use both `@property` and `@abstractmethod`
- Subclasses must provide implementations for these properties

```
class Animal(ABC):
    @property
    @abstractmethod
    def species(self):
        pass

class Dog(Animal):
    @property
    def species(self):
        return "Canine"

dog = Dog()
print(dog.species)  # Output: Canine
```

Source: `https://www.geeksforgeeks.org/abstract-classes-in-python/`

Decorators (22/24)

- Abstract classes with abstract methods cannot be instantiated directly
- Attempting to do so raises a `TypeError`

```
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

# animal = Animal()          # Raises TypeError

class Dog(Animal):
    def make_sound(self):
        return "Woof!"

dog = Dog()                  # Valid
print(dog.make_sound())      # Output: Woof!
```

- Only subclasses implementing all abstract methods can be instantiated

Source: `https://www.geeksforgeeks.org/abstract-classes-in-python/`

Decorators (23/24)

- ABCs are built on top of `ABCMeta`, a metaclass from `abc`
- `@abstractmethod` registers a method as required

```
from abc import ABCMeta, abstractmethod

class MyABC(metaclass=ABCMeta):
    @abstractmethod
    def must_implement(self):
        pass
```

- `ABCMeta` checks for abstract methods at instantiation
- Prevents instantiation of incomplete subclasses

Source: `https://docs.python.org/3/library/abc.html`

Decorators (24/24)

- `@staticmethod`, `@classmethod`, and `@property` are syntactic sugar
- They internally call classes `staticmethod`, `classmethod`, and `property`

```
class MyClass:
    def instance_method(self):
        pass

    static = staticmethod(instance_method)
    class_ = classmethod(instance_method)
    prop = property(instance_method)
```

- Decorators wrap functions using the descriptor protocol
- Python's built-in decorators are implemented as callable classes

Source: `https://docs.python.org/3/howto/descriptor.html`

Variable-Length Arguments in Python

- Python allows functions to accept a variable number of arguments.
- Two general forms:
  - `*args` collects positional arguments into a tuple.
  - `**kwargs` collects keyword arguments into a dictionary.
- Very important when writing decorators.

Understanding *args

- `*args` allows a function to accept any number of positional arguments.
- Inside the function, `args` becomes a tuple.

```
def total(*args):
    print(args)
    return sum(args)

total(1, 2, 3)
# Output: (1, 2, 3)
```

Understanding **kwargs

- **kwargs accepts any number of keyword arguments.
- Inside the function, it becomes a dictionary.

```
def describe(**kwargs):
    print(kwargs)

describe(name="Alice", age=22)
# Output: {'name': 'Alice', 'age': 22}
```

Mixing Positional and Keyword Arguments

- Functions can mix standard parameters, `*args`, and `**kwargs`.
- Correct order: `def f(a, b, *args, **kwargs)`.

```
def info(a, b, *args, **kwargs):
    print(a, b)
    print(args)
    print(kwargs)
```

Example with *args and **kwargs

```
def debug(a, b, *args, **kwargs):
    print("a =", a)
    print("b =", b)
    print("args =", args)
    print("kwargs =", kwargs)

debug(1, 2, 3, 4, x=10, y=20)
# Output:
# a = 1
# b = 2
# args = (3, 4)
# kwargs = {'x': 10, 'y': 20}
```

- This pattern appears often in debugging tools and wrappers.

Why Decorators Use *args and **kwargs

- Decorators wrap functions whose signatures are unknown.
- The wrapper must be able to accept any combination of parameters.

```
def logger(func):
    def wrapper(*args, **kwargs):
        print("Calling:", func.__name__)
        return func(*args, **kwargs)
    return wrapper
```

Decorator Example

```
@logger
def add(x, y):
    return x + y

add(5, 7)
# Output:
# Calling: add
```

- The wrapper must accept `*args` and `**kwargs` because the decorator does not know the function signature.

Transparent Argument Passing

- Decorators should be transparent.
- They should not modify arguments unless explicitly required.

```
def identity_decorator(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
```

Real Use Case: Timing Functions

```
import time

def timing(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print("Time:", time.time() - start)
        return result
    return wrapper
```

Summary

- `*args` collects extra positional arguments.
- `**kwargs` collects extra named arguments.
- Both are essential in decorators.
- They allow writing fully generic wrappers.

What is `@property`?

- `@property` is a built-in decorator for turning a method into a managed attribute.
- It allows attribute access using "dot notation" while executing code behind the scenes.
- Supports reading, validating, computing values dynamically.

Basic @property Example

```
class Person:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        return self._name
```

- `p.name` accesses `p.name()` implicitly.
- No parentheses needed.

Why Use @property?

- Encapsulation without changing the public API.
- Enables:
  - validation
  - computed values
  - backwards compatibility
- Pythonic alternative to getter methods in other languages.

Adding Validation

```
class Student:
    def __init__(self):
        self._age = 0

    @property
    def age(self):
        return self._age

    @age.setter
    def age(self, value):
        if value < 0:
            raise ValueError("Age cannot be negative")
        self._age = value
```

The Setter Method

- The setter is optional.
- Declared with `@propertyname.setter`.
- Allows assignment with syntax:

```
s = Student()
s.age = 20
```

The Deleter Method

- Optional, declared using `@propertyname.deleter`.

```
class Example:
    def __init__(self):
        self._value = 10

    @property
    def value(self):
        return self._value

    @value.deleter
    def value(self):
        print("Deleting value")
        del self._value
```

Read-Only Properties

- If no setter is defined, the property is read-only.

```
class Rectangle:
    def __init__(self, w, h):
        self.w = w
        self.h = h

    @property
    def area(self):
        return self.w * self.h

r = Rectangle(3, 4)
# r.area = 10  -> ERROR
```

Computed Properties

- Properties can compute values dynamically.

```
class Temperature:
    def __init__(self, celsius):
        self.c = celsius

    @property
    def fahrenheit(self):
        return self.c * 9/5 + 32
```

Hiding Internal Implementation

- `@property` allows changing the internal logic without modifying the public interface.

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return round(self._balance, 2)
```

Summary of `@property`

- Turns methods into managed attributes.
- Optional setter and deleter.
- Useful for:
  - validation
  - computed values
  - API stability
  - encapsulation
- Considered Pythonic and widely used in real codebases.

What is a Dataclass in Python?

- A **dataclass** is a Python class designed to store data.
- Introduced in Python 3.7.
- Automatically generates common methods:
  - `__init__()`
  - `__repr__()`
  - `__eq__()`
- Reduces boilerplate code.

Basic Dataclass Example

```python
from dataclasses import dataclass

@dataclass
class Point:
    x: int
    y: int

p = Point(10, 20)
print(p)
# Output: Point(x=10, y=20)
```

- No need to write an `__init__` method.
- The class is clean and readable.

Default Values and Type Hints

- Dataclasses rely on type hints.
- Default values can be set directly.

```
@dataclass
class Student:
    name: str
    id: int = 0
    active: bool = True

s = Student("Alice")
print(s)
```

- Fields without defaults must come first.

Auto-Generated Methods

- Dataclasses generate:
  - `__eq__` — object comparison
  - `__repr__` — readable string representation
- Useful for debugging and testing.

```
a = Point(1, 2)
b = Point(1, 2)
print(a == b)  # True
```

- Normal classes would compare by identity.

Mutable and Immutable Dataclasses

- Dataclasses are mutable by default.
- Setting `frozen=True` makes them immutable (like tuples).

```
@dataclass(frozen=True)
class Color:
    r: int
    g: int
    b: int

c = Color(10, 20, 30)
# c.r = 50  # ERROR: object is immutable
```

- Ideal for constant values and configuration objects.

Generics (1/6)

- Generics allow defining reusable functions/classes that work with multiple data types
- Generics extend the concept of type hints
- Introduced in Python 3.5 with type hints
- Help improve **type safety** and **code clarity**
- Uses `TypeVar` to declare type variables

Type Inference:

- Python is dynamically typed by default
- Generics add optional static typing support

Source: `https://www.tutorialspoint.com/python/python_generics.htm`

Generics (2/6)

- An example of a Generic function:

```python
from typing import List, TypeVar

T = TypeVar('T')          # Define a generic type variable T

def reverse(items: List[T]) -> List[T]:
    # The function takes a list of any type T and returns a reversed list of the
     same type
    return items[::-1]    # Reverse the list using slicing

numbers = [1, 2, 3, 4, 5]
print(reverse(numbers))  # Output: [5, 4, 3, 2, 1]
fruits = ['apple', 'banana', 'cherry']
print(reverse(fruits))   # Output: ['cherry', 'banana', 'apple']
```

- `TypeVar('T')` creates a type placeholder
- Function `reverse` works with any `List[T]`
- Ensures return type matches input list type

Source: `https://www.tutorialspoint.com/python/python_generics.htm`

Generics (3/6)

- An example of a Generic class:

```python
from typing import TypeVar, Generic

T = TypeVar('T')                    # Define a generic type variable T

class Box(Generic[T]):              # Declare a generic class that operates on type T
    def __init__(self, item: T):
        self.item = item            # Store an item of type T
    def get_item(self) -> T:
        return self.item            # Return the stored item of type T

box1 = Box(42)
print(box1.get_item())     # Output: 42
box2 = Box("Hello")
print(box2.get_item())     # Output: Hello
```

- `Box[T]` stores any type
- Type-safe and reusable container
- Output type always matches the input type

Source: `https://www.tutorialspoint.com/python/python_generics.htm`

Generics (4/6)

- `TypeVar` can be constrained to specific allowed types.
- Ensures strict control over the input types.

```python
from typing import TypeVar

T = TypeVar('T', int, float)  # Only int or float allowed

def square(x: T) -> T:
    return x * x

print(square(4))      # OK
print(square(3.5))    # OK
print(square("x"))    # Type checker error
```

- Constrains types without needing a full class hierarchy.

Source: `https://docs.python.org/3/library/typing.html`

Generics (5/6)

- Use ‘`bound=`‘ in `TypeVar` to limit valid types
- Ensures the type variable is a subtype of a specified type

```
from typing import TypeVar

T = TypeVar('T', bound=int)  # T must be a subtype of int

def add(x: T, y: T) -> T:
    return x + y

print(add(2, 3))             # OK: int
print(add_bound(True, 1))    # OK: bool is a subtype of int
print(add("a", "b"))         # Type checker error
```

- Useful to enforce operations like comparison or arithmetic
- Promotes safer and more predictable function behavior

Source: `https://docs.python.org/3/library/typing.html`

Generics (6/6)

- An example of using ‘bound=‘ in a generic class:

```
from typing import TypeVar

class Animal:
    def speak(self) -> str:
        return "Some sound"

class Dog(Animal):
    def speak(self) -> str:
        return "Woof!"

T = TypeVar('T', bound=Animal)  # T must be Animal or subclass

def animal_speak(animal: T) -> str:
    return animal.speak()

dog = Dog()
print(animal_speak(dog))        # Output: Woof!
animal = Animal()
print(animal_speak(animal))     # Output: Some sound
# animal_speak(not an animal)   # Type checker error
```

Inversion of Control (1/8)

- **Inversion of Control (IoC)** is a principle where the control of object creation and management is delegated to an external entity
- **Dependency Injection (DI)** is a technique to achieve IoC by supplying objects with their dependencies from the outside
- Promotes:
  - **Low coupling** – components are independent
  - **High cohesion** – components focus on a single task
  - **Testability and reusability**
- In static-typed languages (e.g. Java), DI is often achieved through complex frameworks
- In Python, DI is easier thanks to its dynamic nature — but still valuable in large applications

Source: `https://python-dependency-injector.ets-labs.org/introduction/di_in_python.html`

Inversion of Control (2/8)

- DI and IoC improve both cohesion and reduce coupling:
  - **Coupling** refers to how tightly components depend on one another
  - High coupling makes systems rigid and difficult to change
  - Low coupling makes systems flexible and easier to test
  - **Cohesion** measures how focused a component is on a single responsibility
  - High cohesion improves maintainability and clarity
  - High cohesion often leads to lower coupling

Source: `https://python-dependency-injector.ets-labs.org/introduction/di_in_python.html`

Inversion of Control (3/8)

Without Dependency Injection:

```
class Service:                          # Classes create their dependencies internally
    def __init__(self):
        self.client = ApiClient()
```

With Dependency Injection:

```
class Service:                          # Dependencies are passed externally
    def __init__(self, client: ApiClient):
        self.client = client
```

- DI increases flexibility and significantly simplifies testing
- You can inject:
  - Mocks or stubs for testing
  - Configured instances for different environments

Source: `https://python-dependency-injector.ets-labs.org/introduction/di_in_python.html`

Inversion of Control (4/8)

- Manual injection can become verbose and scattered:

```
# Application entry point
main(
    service=Service(                                # Inject Service dependency
        client=ApiClient(                           # Inject ApiClient into Service
            api_key=os.getenv("API_KEY"),           # Read API key from environment
            timeout=int(os.getenv("TIMEOUT")), # Read timeout and convert to int
        )
    )
)
```

- The code becomes harder to manage as the app grows
- Duplication of object assembly logic
- Hard to track configuration and dependencies
- **Solution:** A DI framework manages injection centrally and declaratively

Source: `https://python-dependency-injector.ets-labs.org/introduction/di_in_python.html`

Inversion of Control (5/8)

- `dependency-injector` is Python library for DI
- Provides:
  - `Configuration` provider: reads config (e.g. from env)
  - `Singleton`: one shared instance
  - `Factory`: creates new instances on request

```python
from dependency_injector import containers, providers
# Define a container for assembling and configuring dependencies
class Container(containers.DeclarativeContainer):
    config = providers.Configuration()          # Configuration provider
    client = providers.Singleton(               # Singleton provider
        ApiClient,
        api_key=config.api_key,
        timeout=config.timeout
    )
    service = providers.Factory(                # Factory provider
        Service,
        client=client
    )
```

Source: `https://python-dependency-injector.ets-labs.org/introduction/di_in_python.html`

Inversion of Control (6/8)

Using the container to manage dependencies:

```
container = Container()
container.config.api_key.from_env("API_KEY", required=True)
container.config.timeout.from_env("TIMEOUT", as_=int, default=5)
container.wire(modules=[__name__])

@inject
def main(service: Service = Provide[Container.service]):
    ...

main()  # dependencies injected automatically
```

- The container centralizes config and object creation
- `@inject` and `Provide` enable automatic wiring
- Clean and explicit dependency management improves maintainability

Source: `https://python-dependency-injector.ets-labs.org/introduction/di_in_python.html`

Inversion of Control (7/8)

Overriding dependencies for testing:

```
# Replace real ApiClient with a mock during tests
with container.client.override(MockApiClient()):
    main()  # main() runs with mock injected instead of real client
```

- Override let you swap dependencies for testing without changing app code
- Safer and cleaner alternative to monkey-patching internals
- Also useful for environment-specific configurations (dev, staging, prod)

Source: `https://python-dependency-injector.ets-labs.org/introduction/di_in_python.html`

Inversion of Control (8/8)

- DI reduces coupling, increases cohesion, improves testability and clarity
- Python's dynamic typing allows flexible DI but frameworks provide structure and consistency
- Using a DI framework saves time and improves maintainability for large projects
- Testing is easier and more reliable with DI and provider overrides

Source: `https://python-dependency-injector.ets-labs.org/introduction/di_in_python.html`

Pydantic (1/15)

- **Pydantic** is a data validation and settings management library using Python type annotations
- Combines static typing with runtime guarantees

- Python is dynamically typed → little safety at runtime
- Real-world applications need type correctness and structured validation
- Pydantic removes boilerplate and improves code reliability

Source: `https://realpython.com/python-pydantic/`

Pydantic (2/15)

- **Automatic validation**: parses and enforces types
- **JSON serialization**: easy conversion to/from JSON
- **High performance**: core implemented in Rust
- **JSON Schema**: supports schema generation for interoperability

```
pip install pydantic

# Optional email validation
pip install "pydantic[email]"

# Settings management (via .env files)
pip install pydantic-settings
```

- Pydantic v2 is modular — install only what you need
- Email, IPvAnyAddress, etc., are optional extras

Source: `https://realpython.com/python-pydantic/`

Pydantic (3/15)

- An example:

```
from pydantic import BaseModel, EmailStr        # Base class + typed email
from enum import Enum                           # For typed choices
from uuid import uuid4, UUID                    # Unique identifier
from datetime import date                       # Date field

# Define limited choices for department
class Department(Enum):
    HR = "HR"
    IT = "IT"

# Employee model with validation and parsing
class Employee(BaseModel):
    employee_id: UUID = uuid4()                 # Default: generate UUID
    name: str                                   # Required string
    email: EmailStr                             # Valid email format
    date_of_birth: date                         # Parsed from str $\rightarrow$
     date
    salary: float                               # Auto-convertible from str/int
    department: Department                      # Enum validation
    elected_benefits: bool                      # Accepts true/false/1/0
```

Pydantic (4/15)

Pydantic auto-converts input values to the declared types:

```
Employee(
    name="Chris",
    email="chris@example.com",
    date_of_birth="1990-01-01",    # str $\rightarrow$ date
    salary=50000,                  # int $\rightarrow$ float
    department="IT",               # str $\rightarrow$ Department Enum
    elected_benefits=True          # already a bool
)
```

- **Smart coercion:** strings are converted to date, enum, UUID, etc.
- **Validation:** ensures types are correct and values make sense
- Allows flexibility in input (e.g. JSON or user input) without sacrificing safety

Source: `https://realpython.com/python-pydantic/`

Pydantic (5/15)

Invalid input raises `ValidationError` with detailed info:

```
Employee(
    name=False,                     # expected str
    email="not-an-email",           # not a valid email
    date_of_birth="1990-13-40",     # invalid date format
    salary="not-a-float",           # can't parse as float
    department="Marketing",         # not a valid enum value
    elected_benefits="yes"          # expected bool (not "yes")
)
```

- `ValidationError` shows all issues at once (not just the first)
- Each field error includes path, message, and expected type
- Helpful for catching bugs early and returning clean API errors

Source: `https://realpython.com/python-pydantic/`

Pydantic (6/15)

Create model instances from external data:

```
# From a Python dict (e.g. parsed from DB or API)
emp = Employee.model_validate(my_dict)

# From a raw JSON string (e.g. API request body)
emp = Employee.model_validate_json(my_json)
```

- Validates and parses incoming data automatically
- Useful for APIs, configuration files, or DB records
- Ensures typed, clean, validated Python objects

Source: https://realpython.com/python-pydantic/

Pydantic (7/15)

Convert Pydantic models to standard formats:

```
emp.model_dump()        # $\rightarrow$ Python dict
emp.model_dump_json()   # $\rightarrow$ JSON string
```

- Easily serialize models for API responses, logging, or saving to disk
- Converts all fields (including nested models) to plain data
- Automatically handles types like UUIDs, dates, enums, etc.

Source: `https://realpython.com/python-pydantic/`

Pydantic (8/15)

Generate JSON Schema from Pydantic models:

```
Employee.model_json_schema()
```

- Produces a complete JSON Schema describing model structure and validation rules
- Automatically includes field types, enum values, and required vs optional fields
- Helps ensure clients and servers agree on data format

Source: `https://realpython.com/python-pydantic/`

Pydantic (9/15)

- `Field` allows customizing validation and adding metadata to model fields.
- Example customizations:

```python
from pydantic import BaseModel, Field, EmailStr
from uuid import UUID, uuid4

class Employee(BaseModel):
    # default_factory to generate default values
    # frozen=True to make fields immutable after model creation
    employee_id: UUID = Field(default_factory=uuid4, frozen=True)
    # min_length for strings (e.g., name cannot be empty)
    name: str = Field(min_length=1, frozen=True)
    # pattern to enforce regex match (e.g., email domain)
    email: EmailStr = Field(pattern=r".+@example\.com$")
    # alias to rename fields externally
    date_of_birth: date = Field(alias="birth_date", repr=False, frozen=True)
    # gt (greater than) for numeric fields (e.g., salary > 0)
    salary: float = Field(alias="compensation", gt=0, repr=False)
```

Source: `https://realpython.com/python-pydantic/`

Pydantic (10/15)

Invalid data example:

```
incorrect_employee_data = {
    "name": "",                             # is empty
    "email": "user@wrongdomain.com",        # does not match company domain pattern
    "birth_date": "1998-04-02",
    "compensation": -10,                    # must be > 0
    "department": "IT",
    "elected_benefits": True,
}

Employee.model_validate(incorrect_employee_data)
```

Source: https://realpython.com/python-pydantic/

Pydantic (11/15)

- Use `@field_validator` to apply custom validation logic on individual fields
- Example: Enforce employees must be at least 18 years old based on `date_of_birth`

```
from pydantic import field_validator
from datetime import date

class Employee(BaseModel):
    # ... fields as before ...

    @field_validator("date_of_birth")
    @classmethod
    def check_valid_age(cls, dob: date) -> date:
        today = date.today()
        eighteen_years_ago = date(today.year - 18, today.month, today.day)
        if dob > eighteen_years_ago:
            raise ValueError("Employees must be at least 18 years old.")
        return dob
```

Source: `https://realpython.com/python-pydantic/`

Pydantic (12/15)

- Use `@model_validator` to validate the entire model after instantiation
- Useful for validating conditions involving multiple fields
- Example: IT employees can't elect benefits

```
from pydantic import model_validator
from typing import Self

class Employee(BaseModel):
    # ... fields and field validators ...

    @model_validator(mode="after")
    def check_it_benefits(self) -> Self:
        if self.department == Department.IT and self.elected_benefits:
            raise ValueError("IT employees are contractors and don't qualify for
      benefits")
        return self
```

Source: `https://realpython.com/python-pydantic/`

Pydantic (13/15)

- Pydantic enables automatic runtime validation of function arguments using the `@validate_call` decorator

```
from pydantic import validate_call, EmailStr, PositiveFloat, Field
from typing import Annotated
@validate_call
def send_invoice(
    client_name: Annotated[str, Field(min_length=1)],
    client_email: EmailStr,
    items_purchased: list[str],
    amount_owed: PositiveFloat,
) -> str:
    email_str = f"""
    Dear {client_name},
    Thank you for your purchase! You owe ${amount_owed:,.2f} for:
    {items_purchased}
    """
    print(f"Sending email to {client_email}...")
    return email_str
```

Source: `https://realpython.com/python-pydantic/`

Pydantic (14/15)

Valid input example:

```
>>> email_str = send_invoice(
        client_name="Andrew Jolawson",
        client_email="ajolawson@fakedomain.com",
        items_purchased=["pie", "cookie", "cake"],
        amount_owed=20,
    )

Sending email to ajolawson@fakedomain.com...

>>> print(email_str)

Dear Andrew Jolawson,
Thank you for your purchase!
You owe $20.00 for:
['pie', 'cookie', 'cake']
```

Source: `https://realpython.com/python-pydantic/`

Pydantic (15/15)

- Environment variables are commonly used to configure Python apps
- Use `BaseSettings` (from `pydantic-settings`) to:
  - Parse and validate environment variables
  - Apply constraints (e.g., minimum length, URL validation)
  - Integrate with `.env` files

```python
from pydantic import HttpUrl, Field
from pydantic_settings import BaseSettings

class AppConfig(BaseSettings):
    # Must be a valid HTTP/HTTPS URL
    database_host: HttpUrl
    # Must be a string of at least 5 characters
    database_user: str = Field(min_length=5)
    database_password: str = Field(min_length=10)
    api_key: str = Field(min_length=20)
```

Source: `https://realpython.com/python-pydantic/`

Questions?

End of the lectures on the core of Python for Software Engineering.